Credit Scoring Model

Credit Models

José Armando Melchor Soto – 745697
Rolando Fortanell Canedo - 744872
David Campos Ambriz – 7444435

ITESO

Table of Contents

Introduction 3

Data Description 4

1.1 Dataset Dimensions 4

1.2 Description of Variables 5

1.3 Class Distribution of the Target Variable 6

1.4 Variable Types 6

Data Cleaning 7

2.1 Removal of Identification Columns 7

2.2 Cleaning of Textual Noise 8

2.3 Conversion of Numerical Columns 8

2.4 Conversion of Credit_History_Age 8

2.5 Imputation of Categorical Variables 9

2.6 Logical Range Validation 9

2.7 Numerical Variable Imputation 10

2.8 Normalization of Payment_of_Min_Amount 10

2.9 Pipeline Output 11

Feature Analysis 11

3.1 Feature Importance 14

3.1.1 *XGBoost Hyperparameters* 16

3.2 SHAP Analysis 16

Feature Selection 18

Credit Scoring Model 20

5.1 Model Training 21

5.2 Score Construction 21

5.3 Score Normalization and Classification 23

Model Evaluation 25

6.1 Confusion Matrix 26

6.2 Score Distribution Analysis 27

6.2.1 Score Distribution 27

6.2.2 Score Distribution per Class (Combined) 28

6.2.3 Score Distribution per Class 29

6.2.4 Real vs Predicted Score Distributions per Class 31

Assumptions 32

Results and Discussion 33

Limitations 34
Architecture 35
10.1 Project Architecture 35
10.2 Functional Architecture 36
10.3 Object-Oriented Architecture 37
Conclusions 38
Bibliography 40
Appendix 42
Appendix A – Main Script 42
Appendix B – Data Processing 43
Appendix C – Model Training 50
Appendix D – Credit Scoring Pipeline 58
Appendix E – Utilities 64
Appendix F – Visualization 65
Appendix G – Project Configuration 70

Introduction

The global financial system and the economy as we know depend internally on many sophisticated processes working within each financial institution to evaluate and make decisions regarding their movements. Credit evaluation through credit scoring models and its introduction in 1960 established new standards for risk management in credit institutions, transitioning from subjective judgment toward rigorous methodologies. In this way, a point has been reached where the primary objective of a credit scoring model, rather than mere statistical classification, is a system where the lender must deduce the credit quality of the individual requesting it by applying indicators of behavior and solvency.

In contemporary banking, although traditional credit models inspired by Altman's work may continue to be used in some cases , some have evolved in an attempt to capture patterns that were previously ignored and that can now serve to obtain better models and thereby capture more clients.

Under new guidelines, such as those from the Basel Committee on Banking Supervision (2005), the care with which models are constructed requires validation that is not only statistical and predictive but also ensures that each of its parameters possesses an underlying economic justification. In this sense, models of this type have been used as a translation of a person's financial history and leverage level into a risk metric that is easy to interpret.

Based on this, the present project seeks to develop an integral credit model built with the objective of solving a multiclass classification problem through a feature importance analysis. Within it, priority will be given to obtaining an architecture in which importance is assigned to characteristics that can explain an individual's behavior and liquidity in a more realistic and precise manner. Thus, a score will be assigned, and they will be classified according to their risk rating into one of three credit categories: “Good”, “Standard”, and “Poor”.

Data Description

The dataset used in this project (train-3.csv) corresponds to a Credit Score Classification dataset obtained from Kaggle. It contains financial and demographic information for approximately 100,000 records of banking customers.

The objective of the dataset is to predict an individual's credit score classification into three ordinal categories: Poor, Standard, and Good. Therefore, it represents a supervised multiclass classification problem.

Each record represents a customer's monthly information and includes variables related to payment behavior, level of indebtedness, credit history, usage of financial products, and income.

The dataset is designed for the training and evaluation of supervised classification models in the context of consumer credit risk assessment.

1.1 Dataset Dimensions

In its raw form, the dataset has the following structure:

- Observations: 100,000 records
- Initial variables: 28 columns, including identification fields and the target variable.
- Post-preprocessing variables: 40 total columns, composed of:
    - 23 core feature variables
    - 17 binary indicator variables generated during data cleaning.
- Target variable: *Credit_Score* (Poor / Standard / Good)

The increase in dimensionality after preprocessing is due to the creation of binary indicator variables designed to capture missing values and logically inconsistent observations. Rather than discarding these records, this approach preserves potentially informative patterns that may contribute to predictive performance during model training.

1.2 Description of Variables

Table I presents the complete variable dictionary of the dataset. Personal identification columns (*ID, Customer_ID, Name, SSN,* and *Month*) were excluded from the analysis, as they do not provide predictive value for the modeling process and contain personally identifiable information (PII).

Variable	Type	Description	Range / Values
Age	Numerical	Age of the account holder	18 – 100
Occupation	Categorical	Customer's professional occupation	16 categories
Annual_Income	Numerical	Declared annual income (USD)	> 0
Monthly_Inhand_Salary	Numerical	Net monthly take-home salary (USD)	> 0
Num_Bank_Accounts	Numerical	Number of active bank accounts	0 – 20
Num_Credit_Card	Numerical	Number of credit cards	0 – 50
Interest_Rate	Numerical	Average interest rate on loans (%)	0 – 100
Num_of_Loan	Numerical	Number of active loans	0 – 50
Type_of_Loan	Categorical	Types of contracted loans	Multi-label
Delay_from_due_date	Numerical	Average number of days past due	—
Num_of_Delayed_Payment	Numerical	Number of recorded delayed payments	≥ 0
Changed_Credit_Limit	Numerical	Percentage change in credit limit	—
Num_Credit_Inquiries	Numerical	Number of recent credit inquiries	≥ 0
Credit_Mix	Categorical	Quality of the credit mix	Good / Standard / Bad
Outstanding_Debt	Numerical	Total outstanding debt (USD)	≥ 0
Credit_Utilization_Ratio	Numerical	Credit utilization ratio (%)	0 – 100
Credit_History_Age	Numerical	Length of credit history (years)	> 0
Payment_of_Min_Amount	Binary	Indicates whether only the minimum payment is made	Yes / No
Total_EMI_per_month	Numerical	Total monthly installment payments (EMI) (USD)	≥ 0
Amount_invested_monthly	Numerical	Amount invested monthly (USD)	≥ 0
Payment_Behaviour	Categorical	Payment behavior pattern	6 categories
Monthly_Balance	Numerical	Available monthly balance (USD)	—
Credit_Score	Categorical (Target)	Customer's credit score classification	Poor / Standard / Good

Table I : Dictionary of the Dataset

1.3 Class Distribution of the Target Variable

Table II summarizes the distribution of the target variable Credit_Score. The dataset exhibits a moderate class imbalance. The Standard category is the majority class, representing 53.17% of the observations. The Poor class accounts for 29.00%, while Good is the least represented class with 17.83% of the total records.

Figure 1 visually reinforces this imbalance, showing a clear dominance of the Standard category over the other two classes. Although the imbalance is not extreme, it is sufficiently pronounced to potentially influence model performance, particularly if evaluation relies solely on accuracy.

For this reason, performance metrics that account for class imbalance (such as macro average F1-score or per-class ROC-AUC) are more appropriate for model assessment.

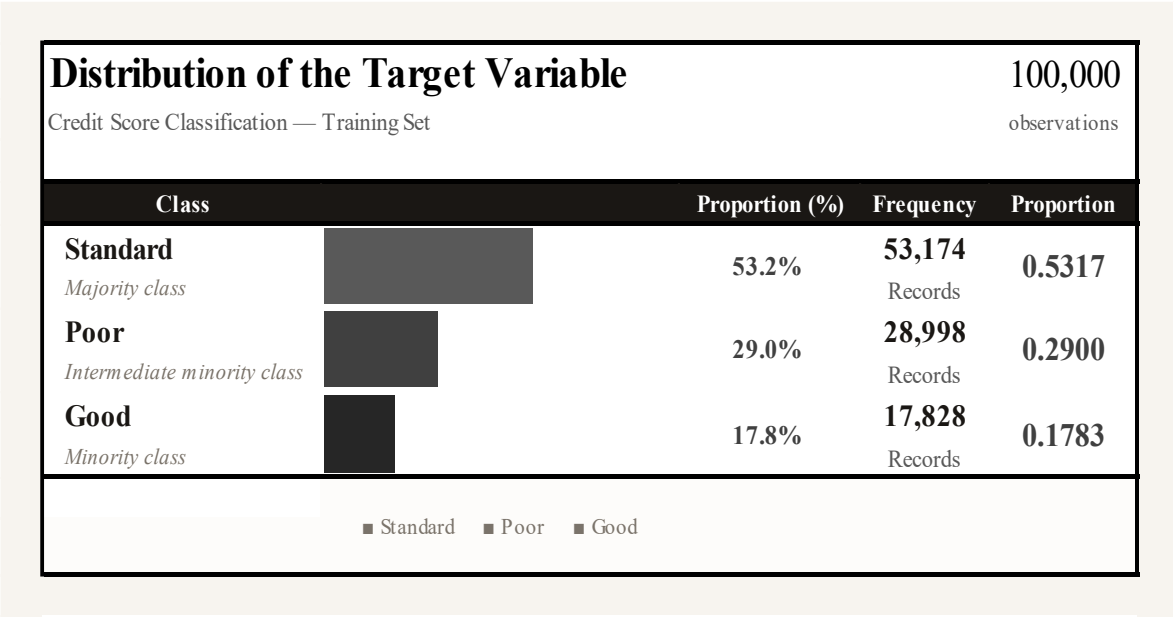

Figure II : Distribution of the Target Variable (Credit_Score).

1.4 Variable Types

The dataset variables can be grouped into three main categories:

- Numerical variables (12): *Annual_Income, Monthly_Inhand_Salary, Changed_Credit_Limit, Outstanding_Debt, Credit_Utilization_Ratio,*

Credit_History_Age, Total_EMI_per_month, Amount_invested_monthly, Monthly_Balance, Delay_from_due_date, Num_Credit_Inquiries, and Num_of_Delayed_Payment.

- Discrete numerical variables (5): *Age, Num_Bank_Accounts, Num_Credit_Card, Interest_Rate, and Num_of_Loan.*
- Categorical variables (5): *Occupation, Type_of_Loan, Credit_Mix,Payment_of_Min_Amount, and Payment_Behaviour.*

It is important to note that several numerical variables were originally stored as object (string) types due to the presence of noise characters. These required explicit cleaning and type conversion during preprocessing

Data Cleaning

The data cleaning process was implemented through a modular preprocessing pipeline in Python, encapsulated within the data_preprocessing() function. The pipeline was designed with a traceability-oriented approach, ensuring that each potentially destructive transformation is accompanied by the creation of binary indicator variables. These indicators capture information about missing values and logically inconsistent observations, allowing the model to retain potentially informative signals that would otherwise be lost during the cleaning process.

2.1 Removal of Identification Columns

As an initial step, five personal identification columns were removed, as they do not provide predictive value for the model: *ID, Customer_ID, Month, Name*, and *SSN.*

Removing these variables reduces unnecessary dimensionality and prevents potential information leakage while also mitigating privacy concerns associated with personally identifiable information (PII).

2.2 Cleaning of Textual Noise

Systematic corruption patterns were detected across several columns originally stored as strings. The identified noise included residual underscores (e.g., "28_", "34_", "34847.84_") and special character sequences such as "!@9#%8".

These patterns were removed using regular expressions applied globally across the DataFrame. Any fields that became empty after this cleaning process were converted to NaN values for subsequent handling.

This corruption primarily affected 11 numerical columns, as well as the categorical variable *Credit_Mix*, where the placeholder value "_" appeared in approximately 20.2% of the records.

2.3 Conversion of Numerical Columns

After removing textual noise, the columns listed in STRIP_NOISE_COLS were converted to numerical format (float64) using pd.to_numeric(errors='coerce'). This method assigns NaN to any value that cannot be successfully parsed as a numeric value.

The affected columns include:

Age, Annual_Income, Num_of_Loan, Num_of_Delayed_Payment, Changed_Credit_Limit, Outstanding_Debt, Amount_invested_monthly, Monthly_Balance, Num_Credit_Inquiries, Total_EMI_per_month, and *Credit_Utilization_Ratio.*

2.4 Conversion of Credit_History_Age

The variable Credit_History_Age was originally stored as a textual string in the format “22 Years and 3 Months”. This representation was transformed into decimal years using the function _convert_credit_history_age(), which extracts the numerical components through regular expression matching.

The conversion follows the formula:

$$credit_history_age = years + (months / 12)$$

Missing values prior to the transformation were preserved as NaN. After conversion, the resulting values range from 0.08 to 33.67 years. This approach retains the ordinal information of the variable while representing it in a continuous numerical format directly usable by machine learning models.

2.5 Imputation of Categorical Variables

Categorical variables containing missing values were imputed using the explicit label "Missing". This strategy creates an additional category that allows the model to capture potential patterns associated with the absence of information, rather than discarding observations or implicitly imputing values. The affected columns include Credit_Mix, Occupation, Type_of_Loan, and Payment_Behaviour.

2.6 Logical Range Validation

Business rules were applied to detect values that were numerically valid but logically implausible. For each affected variable, values falling outside the defined bounds were replaced with NaN, and a binary indicator column <variable>_invalid was created to preserve information about the original anomaly.

Table III summarizes the validation rules applied during preprocessing.

Variable	Minimum	Maximum	Action
Age	18	100	Out-of-range values → NaN + Age_invalid indicator
Num_of_Loan	0	50	Out-of-range values → NaN + indicator
Num_Bank_Accounts	0	20	Out-of-range values → NaN + indicator
Num_Credit_Card	0	50	Out-of-range values → NaN + indicator
Interest_Rate	0	100	Out-of-range values → NaN + indicator
Credit_History_Age	0.01	—	Lower bound only applied

Table III : Range validation rules applied during preprocessing.

2.7 Numerical Variable Imputation

For all numerical columns containing missing values, median imputation was applied. The median was preferred over the mean due to its robustness to skewed distributions and extreme outliers, which are common in financial variables such as Annual_Income (max: $24,198,062) and Total_EMI_per_month (max: $82,331). Additionally, each imputed column generates a binary indicator <variable>_missing to preserve information about the original absence of data.

Table IV summarizes the variables with the highest proportion of missing values and the applied strategy.

Variable	Missing Values	% of Total	Strategy Applied
Monthly_Inhand_Salary	15002	15.00%	Median imputation
Type_of_Loan	11408	11.41%	"Missing" category
Credit_Mix	20195	20.20%	"Missing" category (placeholder "_")
Credit_History_Age	9030	9.03%	Median imputation
Num_of_Delayed_Payment	7002	7.00%	Median imputation
Amount_invested_monthly	4479	4.48%	Median imputation
Num_Credit_Inquiries	1965	1.97%	Median imputation
Monthly_Balance	1200	1.20%	Median imputation

Table IVII: Summary of missing values and imputation strategy.

2.8 Normalization of Payment_of_Min_Amount

An inconsistency was detected in the encoding of the variable *Payment_of_Min_Amount*, where the value "NM" (Not Mentioned or a data entry artifact) appeared alongside the valid values "Yes" and "No". This inconsistency was corrected by replacing "NM" with "No", thereby standardizing the variable into a clean binary representation.

2.9 Pipeline Output

After applying the complete preprocessing pipeline, the cleaned dataset exhibits the following characteristics:

- 0 missing values across all columns (complete imputation verified).
- 23 original feature columns containing relevant information (PII removed and correct data types enforced).
- 17 additional binary indicator columns (_invalid and _missing) to preserve data quality information.
- Final dataset dimensions: 100,000 × 40.
- Credit_History_Age converted to decimal years (range: 0.08 – 33.67 years).
- Payment_of_Min_Amount normalized to a binary representation: Yes / No.

The preprocessing pipeline was designed to be reproducible and extensible. The save_path parameter allows the cleaned dataset to be saved directly from the function, facilitating its integration into automated machine learning workflows.

Feature Analysis

To ensure that the development of the credit scoring model does not remain purely a statistical exercise, mathematical tools were applied to obtain an objective quantification of the borrower's risk profile as well as an estimation of the probability of default. This approach is inspired by early corporate credit risk classification models, which relied on specific financial metrics to capture a firm's liquidity, profitability, and leverage conditions (Altman, 2018).

As previously discussed, this project interprets the dataset as an ecosystem composed of indicators of financial stress rather than as a collection of isolated variables. Under this perspective, the available features were analyzed and categorized into three main dimensions.

Repayment capacity and leverage level

This category includes metrics that describe the ability of a borrower to absorb and repay obligations while considering existing debt levels. Variables such as *Outstanding_Debt* and the total monthly installment payment *Total_EMI_per_month* provide insight into an individual's indebtedness and available monthly liquidity.

Payment behavior and credit history

This dimension groups variables that help infer the financial discipline and historical credit behavior of a borrower, which in turn allows a statistical approximation of their future financial behavior. Metrics such as payment delay *Delay_from_due_date*, minimum payment behavior *Payment_of_Min_Amount_Yes*, and credit diversification indicators *Credit_Mix_Standard* and *Credit_Mix_Good* are used to evaluate the borrower's payment patterns.

Customer conditions and risk perception

This category captures attributes derived from previous internal risk assessments of the borrower. Information such as the assigned interest rate *Interest_Rate*, the number of active credit cards *Num_Credit_Card*, and credit limit modifications *Changed_Credit_Limit* may indicate reliance on multiple credit lines and can serve as early warning signals for identifying higher-risk borrower profiles (World Bank Group, 2019).

Additionally, to ensure a certain level of mathematical robustness in the preliminary phase of variable selection for the linear scoring model similarly to Altman's multiple discriminant analysis approach an exploratory analysis was conducted to approximate the statistical assumption of independence between variables. For this purpose, covariance and correlation matrices were analyzed.

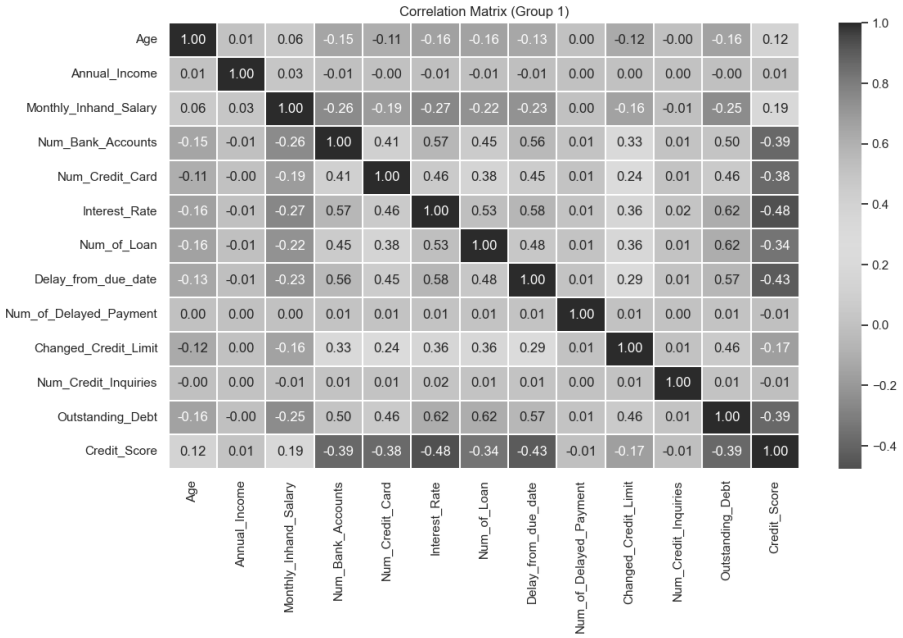

Figure II. a: Correlation Matrix (All variables)

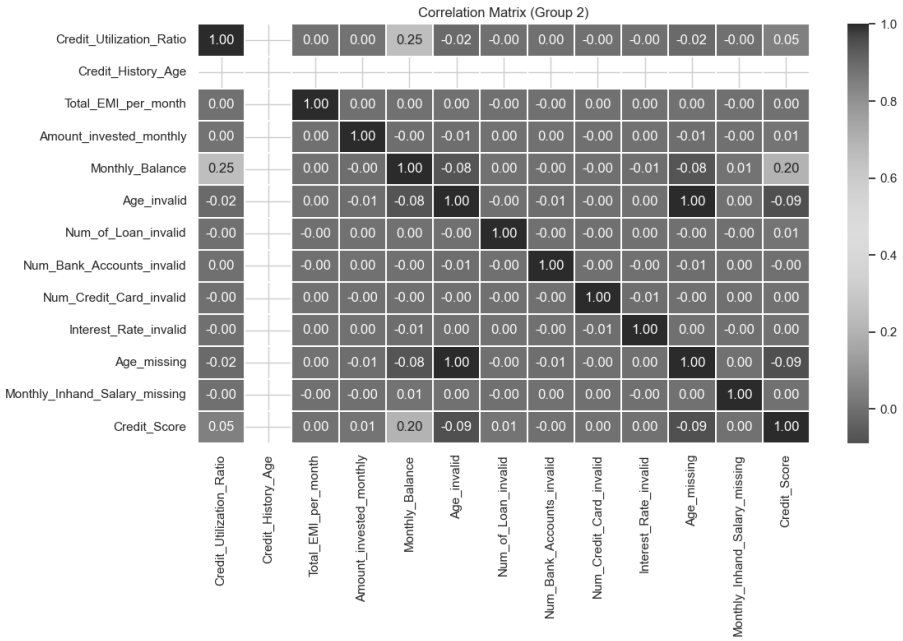

Figure IV. b: Correlation Matrix (All variables)

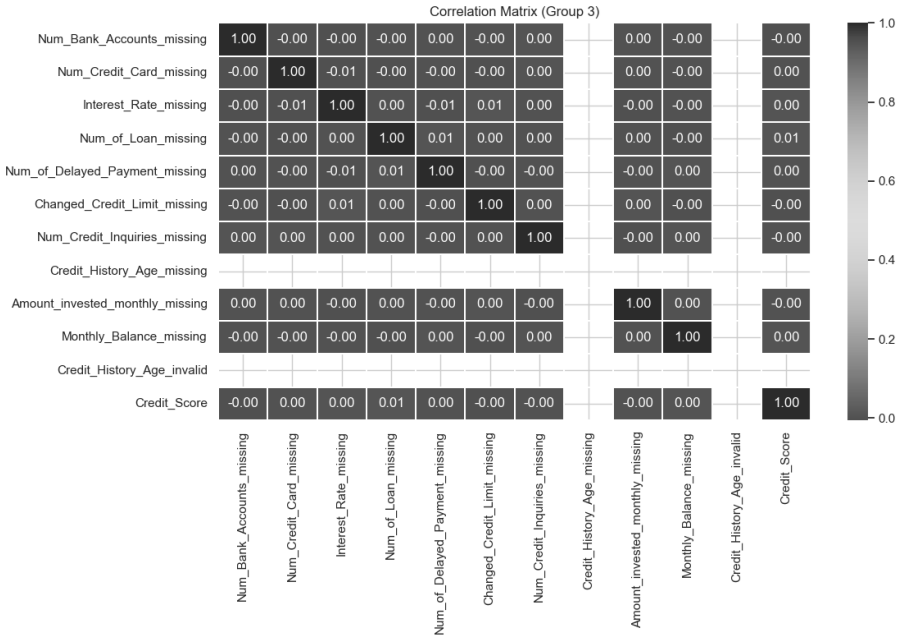

Figure VII. c: Correlation Matrix (All variables)

This process aims to identify variables that capture independent aspects of the borrower's financial profile while discarding redundant variables or those with little or no correlation before applying classification algorithms that validate the importance or statistical relevance of certain predictors in relation to the target variable *Credit_Score*. For instance, variables shown in figures I. b and I. c could potentially be discarded since none exhibit a significant correlation with the target. Conversely, variables such as *Interest_Rate*, *Delay_from_due_date*, and *Outstanding_Debt*, among others, could be considered relevant candidates for the model.

3.1 Feature Importance

To mitigate potential human bias in manual variable selection and to eliminate variables that may introduce noise into the model, a filtering process using the gradient boosting decision tree algorithm XGBoost was implemented as an additional validation step to identify the most relevant predictors.

In credit risk modeling, ensemble methods such as XGBoost have proven highly effective in identifying complex patterns and nonlinear relationships between variables. Furthermore, these models quantify the information gain associated with each feature, allowing an

assessment of how strongly each variable contributes to the classification of borrowers into different risk categories (Yotsawat, 2021). By applying this methodology, which iteratively evaluates the predictive contribution of each variable as a statistical filter it was possible to isolate the predictors that provide the highest mathematical significance in the classification of the *Credit_Score*.

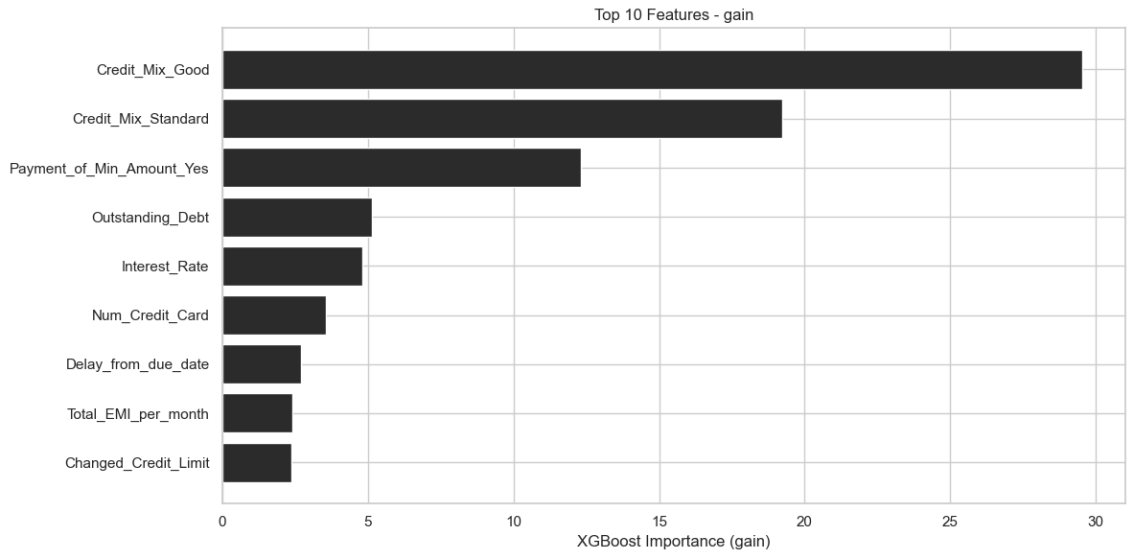

Figure IVIII: Feature Importance calculated by XGBoost

Although the results presented in the previous figure provide a useful indication of which variables may be suitable for inclusion in the final model, it is important to note that it features importance metrics only measure the statistical contribution of each variable and do not capture the direction of the relationship with the target variable. In other words, a high level of importance does not necessarily guarantee an economically meaningful or interpretable relationship. Therefore, it is necessary to confirm the direction and economic consistency of these predictors so that their inclusion in the final model can be justified and interpreted within a financially meaningful framework.

3.1.1 *XGBoost Hyperparameters*

XGBoost Hyperparameters		
Parameter	Value	Description
objective	multi:softprob	Defines a multi-class classification objective and outputs class probabilities.
num_class	3	Specifies the number of classes in the credit score prediction problem.
n_estimators	1000	Number of trees used in the boosting process. A larger number helps stabilize feature importance estimates.
max_depth	6	Maximum depth of each tree, controlling model complexity and reducing overfitting.
learning_rate	0.05	Controls how much each tree contributes to the model. A smaller value leads to more gradual learning.
subsample	0.6	Fraction of training observations randomly sampled for each tree to reduce variance.
colsample_bytree	0.8	Fraction of features randomly sampled for each tree to avoid reliance on a small subset of predictors.
eval_metric	mlogloss	Evaluation metric used during training for multi-class classification.
random_state	42	Ensures reproducibility of the results.

Table IV : XGBoost Hyperparams Used for Feature Importance

An XGBoost classifier was trained using the hyperparameters shown below. These parameters were selected to balance model complexity and stability of the importance estimates. A relatively large number of trees combined with a small learning rate allows the model to learn gradually, while subsampling techniques help reduce variance and prevent overfitting.

The results indicate that a subset of variables contributes significantly more to the prediction of the credit score. These predictors were therefore considered strong candidates for the final model, as they provide the highest information gain in the classification process.

3.2 SHAP Analysis

To interpret the relationships identified by the model, the Shapley Additive Explanations (SHAP) framework was implemented as a tool to provide interpretability to the selected variables. This technique decomposes the predictions of the XGBoost model into additive contributions of each feature, allowing the exact impact and direction of each variable on the final risk classification to be observed for every observation (Gramegna & Giudici, 2021).

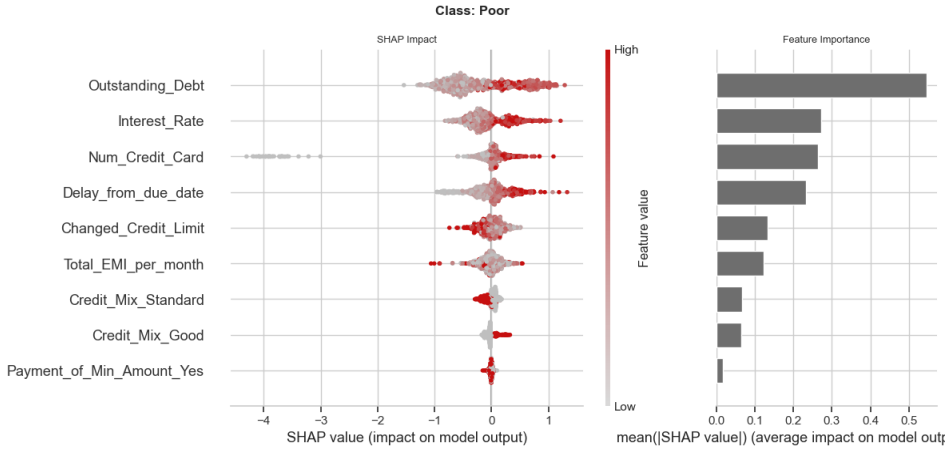

Figure IV. a : SHAP Impact for each variable (Poor Class).

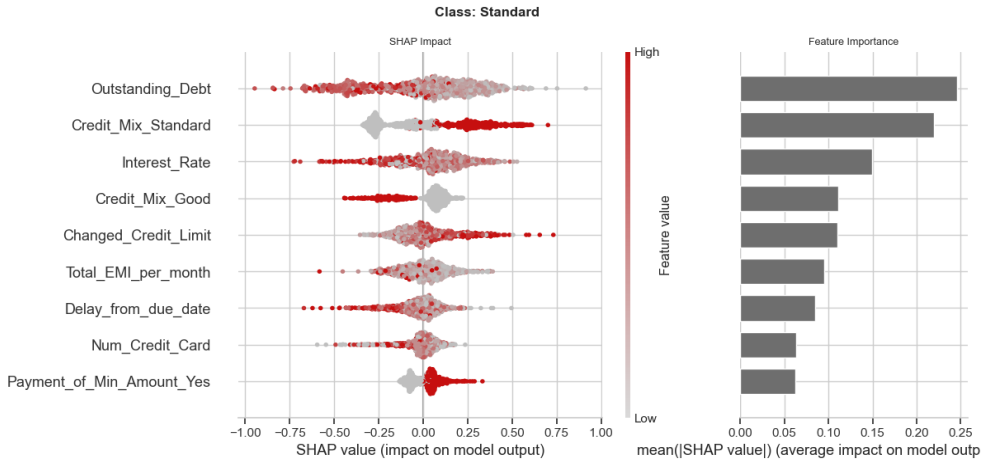

Figure IV. b : SHAP Impact for each variable (Standard Class)

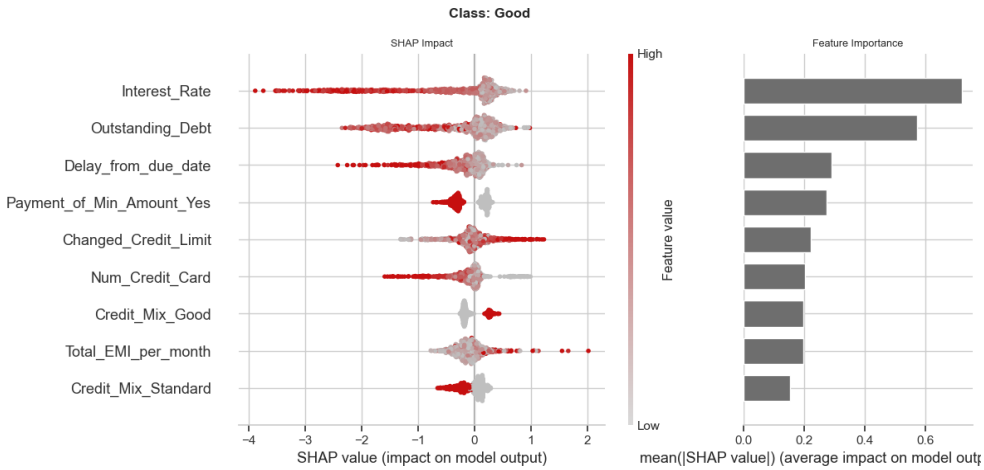

Figure IV. c : SHAP Impact for each variable (Good Class).

As shown in the distribution of SHAP values for the highest risk category "Poor" (Figure III. a), variables such as *Outstanding_Debt* and *Delay_from_due_date* exhibit an inverse relationship with financial solvency. Higher values of these variables generate a significant penalty in the model, with red points representing higher feature values concentrated toward the right side of the plot, indicating a stronger contribution to the probability of belonging to higher-risk categories. In contrast, within the "Good" class, these higher values tend to appear toward the left side of the distribution, reflecting the opposite effect on classification.

Similarly, the model suggests that both "Poor" and "Standard" borrowers tend to exhibit a credit diversification pattern more heavily weighted toward the standard credit mix. In contrast, the "Good" borrower group shows greater variability in credit diversification, without a dominant pattern, although slightly tilted toward higher levels of *Credit_Mix_Good.*

Furthermore, within the "Good" classification group, the variable *Payment_of_Min_Amount_Yes* appears to exert a negative contribution to higher-risk classifications. This suggests that making payments above the required minimum may help borrowers maintain a more stable and lower-risk credit profile.

Feature Selection

After completing the theoretical and empirical validation stages through feature analysis, feature importance, and SHAP analysis, a final selection of nine variables was determined for the construction of the credit scoring model. This selection aims to maintain a balanced representation of the three conceptual pillars previously described.

Repayment capacity and leverage level

The following variables were selected:

- *Outstanding_Debt*
- *Total_EMI_per_month*

Similarly to corporate risk models, where leverage levels are interpreted as key predictors of financial solvency (Altman, 2000), in the context of personal credit the total debt balance

combined with monthly installment payments can constrain consumer liquidity. High levels of indebtedness may reduce an individual's ability to withstand periods of financial stress and absorb unexpected economic shocks, thereby significantly increasing the probability of default (Consumer Financial Protection Bureau [CFPB], 2024).

Payment behavior and credit history

The following variables were selected:

- *Delay_from_due_date*
- *Payment_of_Min_Amount_Yes*

According to Avery et al. (2003) and Thomas (2000), payment behavior is often one of the most robust predictors of future credit risk. Frequent delays in meeting payment obligations or consistently making only the minimum required payment are strong indicators of short-term liquidity stress. These patterns may reflect an inability to fully service financial obligations and are therefore commonly interpreted by credit risk models as early warning signals of potential default (Board of Governors of the Federal Reserve System, 2007).

Customer conditions and risk perception

The following variables were selected:

- *Credit_Mix_Standard*
- *Credit_Mix_Good*
- *Interest_Rate*
- *Num_Credit_Card*
- *Changed_Credit_Limit*

First, credit portfolio diversification captured through *Credit_Mix_Standard* and *Credit_Mix_Good* provides information regarding the borrower's ability to manage multiple forms of credit simultaneously. Successfully managing different types of credit obligations can be interpreted as a signal of financial discipline and borrower credibility (Consumer Financial Protection Bureau [CFPB], 2024).

Additionally, the inclusion of Interest_Rate follows the economic principle of risk-based pricing, which states that financial institutions typically charge higher interest rates to higher-risk borrowers to compensate for the increased probability of default (Kagan & Khartit, 2020). In this sense, the assigned interest rate can act as an indirect signal of the risk perception already formed by other financial institutions (World Bank Group, 2019).

Similarly, the variable *Num_Credit_Card* may indicate exposure to short-term credit obligations. While holding multiple credit cards may arise from legitimate consumer needs, in some cases it may also reflect structural reliance on credit as a mechanism to finance everyday expenses, which may indicate financial fragility (Avery et al., 2003; Board of Governors of the Federal Reserve System, 2007).

Finally, *Changed_Credit_Limit* was included to capture adjustments made by financial institutions in response to changes in the borrower's risk profile. Credit limit modifications often reflect internal reassessments of the borrower's financial situation, where institutions may either expand or restrict credit availability depending on perceived changes in repayment capacity (Board of Governors of the Federal Reserve System, 2007).

Credit Scoring Model

Once the exploratory variable analysis, statistical validation through feature importance, and the interpretation of each predictor's impact using SHAP values were completed, the final credit scoring model was developed.

The objective of this stage of the project is to transform the previously selected variables into an interpretable scoring system capable of classifying customers into different levels of credit risk. Unlike the exploratory stage, where ensemble models such as XGBoost were used to identify nonlinear relationships and evaluate feature relevance, this stage adopts a more classical and transparent modeling approach through a multiclass logistic regression.

The choice of this model is motivated by the fact that ensemble methods, although often highly predictive, tend to be more difficult to interpret. In contrast, linear models such as logistic regression provide a more parsimonious and transparent framework, allowing a

clearer interpretation of the relationship between the explanatory variables and the probability of belonging to each credit risk category (Thomas, 2000).

Specifically, the multiclass logistic regression model allows the estimation of the probability that an individual belongs to each of the risk categories defined in the target variable *Credit_Score*: Poor, Standard, and Good. These models have been widely adopted in the financial industry due to their interpretability, statistical stability, and their ability to be translated into practical scoring systems used in credit decision-making processes (Hand & Henley, 1997; Thomas, Crook & Edelman, 2017).

The model was trained using the nine variables previously selected during the feature selection stage, which represent different aspects of the customer's financial profile, including their level of indebtedness, payment behavior, and credit history.

5.1 Model Training

The model was trained by integrating both the preprocessing stage and the model estimation within a unified framework. During the preprocessing phase, the numerical variables were standardized using *StandardScaler*, which allows the variables to be normalized to a common scale and improves the numerical stability in the estimation of the model's coefficients and weights (James et al., 2021).

After preprocessing, the logistic regression model was fitted using the variables previously selected during the feature selection stage. To preserve the original distribution of the classes within the dataset, the data was partitioned using stratified sampling, allocating 80% of the observations to the training set and the remaining 20% to the test set (James et al., 2021).

5.2 Score Construction

Once the logistic regression model has been trained, the estimated coefficients can be used to construct an interpretable linear scoring function. In general, logistic regression models rely on a linear combination of the explanatory variables and their corresponding coefficients, which can be expressed as:

$$z = wX + b$$

where X represents the vector of explanatory variables, w the vector of estimated coefficients, and b the intercept term of the model. This linear component is commonly referred to as the logit, which represents the log-odds associated with a particular class in a logistic regression model (James et al., 2021).

In a multiclass logistic regression setting, a separate logit is estimated for each class. Therefore, the model produces three linear functions corresponding to the credit categories *Poor*, *Standard*, and *Good*:

$$logit_{Poor} = w_{Poor}X + b_{Poor}$$
$$logit_{Standard} = w_{Standard}\,X + b_{Standard}$$
$$logit_{Good} = w_{Good}\,X + b_{Good}$$

More generally, the logit for any class k can be expressed as a linear function of the explanatory variables:

$$logit_k = w_k X + b_k$$

where k represents each of the possible credit risk categories.

To construct a single interpretable credit score that reflects the relative difference between low-risk and high-risk borrowers, the scoring function is defined as the difference between the logits associated with the *Good* and *Poor* classes:

$$Score = logit_{Good} - logit_{Poor}$$

Substituting the linear expressions of each logit results in:

$$Score = (w_{Good}X + b_{Good}) - (w_{Poor}X + b_{Poor})$$

which simplifies to another linear scoring function:

$$Score = (w_{Good} - w_{Poor})X + (b_{Good} - b_{Poor})$$

This formulation preserves the linear structure of traditional credit scoring models while capturing the relative separation between low-risk and high-risk borrower profiles. The resulting score therefore mañitas the general form:

$$Score = wX + b$$

making it possible to directly interpret the contribution of each variable to the final credit score (Thomas, Crook & Edelman, 2017).

The score is therefore constructed as the difference between the logits of the *Good* and *Poor* classes, capturing the relative separation between low-risk and high-risk borrower profiles. The *Standard* category naturally emerges as an intermediate region defined by the score thresholds applied to the continuous score.

5.3 Score Normalization and Classification

After computing the linear score, the resulting values are transformed into a normalized scoring scale to facilitate interpretation and comparison across observations. In credit scoring systems, it is common practice to map the raw model output into a bounded numerical range that can be interpreted as a standardized credit score (Thomas, Crook & Edelman, 2017).

To achieve this, the linear score is rescaled using a min–max normalization approach, which maps the raw score values into a predefined interval. This transformation preserves the relative ordering of observations while converting the model output into a more interpretable scoring scale.

Formally, the normalized score can be expressed as:

$$Score_{norm} = \frac{Score - Score_{min}}{Score_{max} - Score_{min}}$$

where $Score_{min}$ and $Score_{max}$ represent the minimum and maximum values of the computed score within the dataset.

This normalization step ensures that the resulting credit score is easier to interpret and can be consistently applied across different datasets.

Once the normalized credit score has been computed, observations are classified into predefined credit risk categories using score thresholds. These thresholds partition the continuous score into three distinct regions corresponding to the credit classes:

- Poor
- Standard
- Good

Lower score values indicate a higher level of credit risk, while higher scores represent more financially stable borrowers. The intermediate region corresponds to the Standard category, which represents borrowers with moderate credit risk.

In the proposed model, the classification thresholds were set at $t_1 = 327$ and $t_2 = 409$, which divide the credit score distribution into three intervals:

$$Score < 327 \rightarrow Poor$$
$$327 \leq Score < 409 \rightarrow Standard$$
$$Score \geq 409 \rightarrow Good$$

This threshold-based classification allows the continuous credit score to be translated into discrete credit categories, facilitating its use in practical credit decision processes. Using these thresholds, the model achieved an accuracy of 0.623 on the test dataset.

Model Evaluation

To evaluate the predictive performance of the proposed credit scoring model, the classification accuracy metric was used. Accuracy measures the proportion of correctly classified observations relative to the total number of observations in the dataset (James et al., 2021).

Formally, accuracy can be expressed as:

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of observations}}$$

The evaluation was conducted using the test dataset, which represents 20% of the original observations and was not used during the training stage. Evaluating the model on unseen data provides a more reliable estimate of its ability to generalize to new observations.

Predictions were obtained by computing the credit score for each observation and assigning a risk category according to the predefined score thresholds. The predicted classes were then compared with the true labels of the dataset to determine the proportion of correctly classified observations.

Using this approach, the model achieved an accuracy of 0.623, indicating that approximately 62.3% of the observations were correctly classified into the credit risk categories *Poor*, *Standard*, and *Good*.

6.1 Confusion Matrix

Training Set

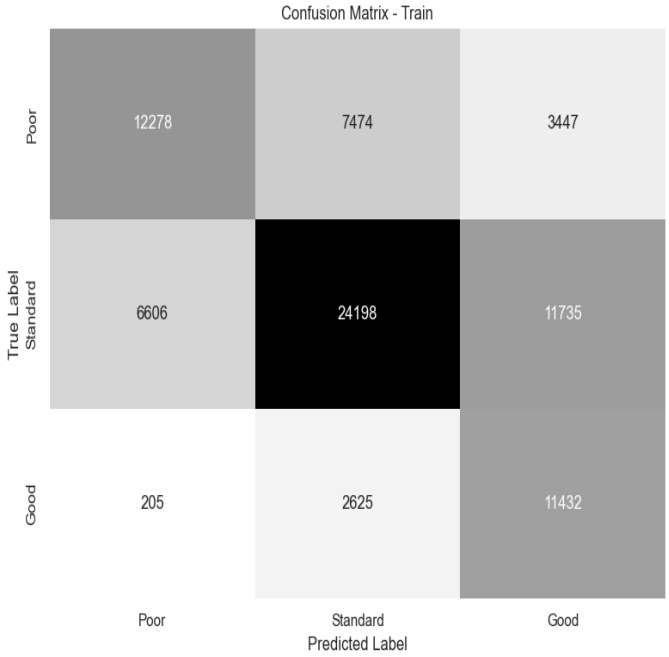

Figure V. a : Confusion Matrix - Train

The confusion matrix for the training dataset shows that the model correctly classifies a large portion of observations within the Standard category, which is expected since this class represents the largest share of the dataset. Some misclassification occurs between the Poor and Standard categories, likely because borrowers in these groups may exhibit similar financial characteristics, resulting in overlapping score ranges that make them harder to distinguish.

Test Set

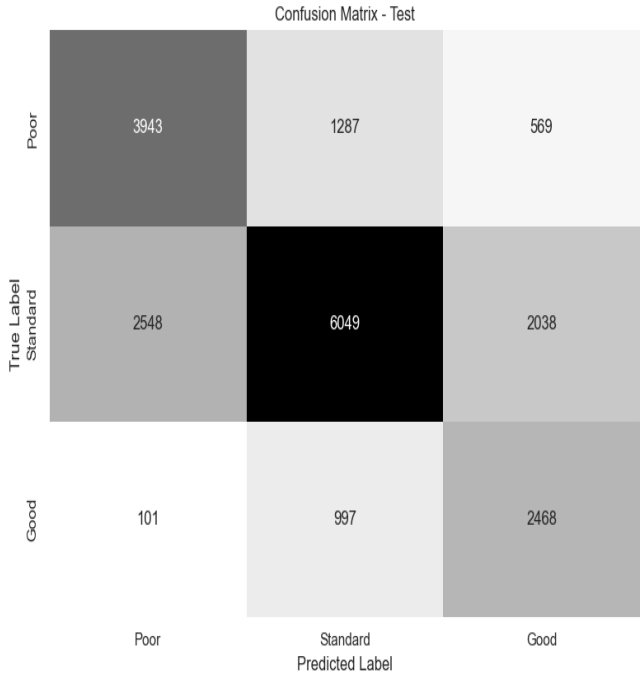

Figure V. b : Confusion Matrix - Test

The confusion matrix for the test dataset shows a similar pattern to the training set, with the model performing best in identifying the Standard category. Misclassifications mainly occur between the Poor and Standard classes, which may be explained by the intermediate risk profiles present in the dataset. These borrowers may share financial characteristics that place them near the classification thresholds, making them more difficult to separate perfectly.

6.2 Score Distribution Analysis

6.2.1 Score Distribution

Training Set

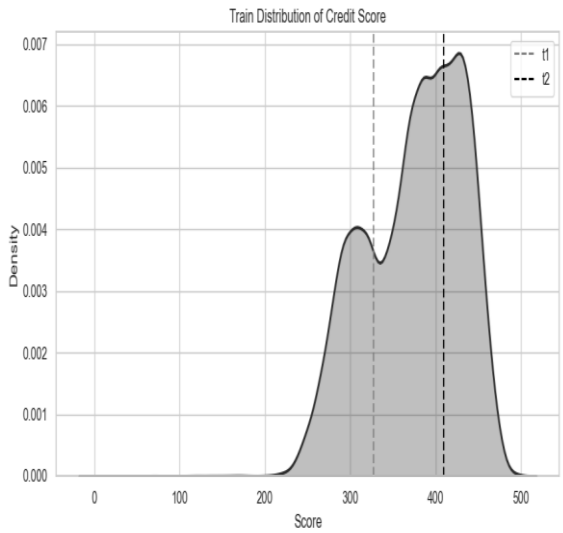

Figure VI. a : Distribution of Credit Score - Train

Test Set

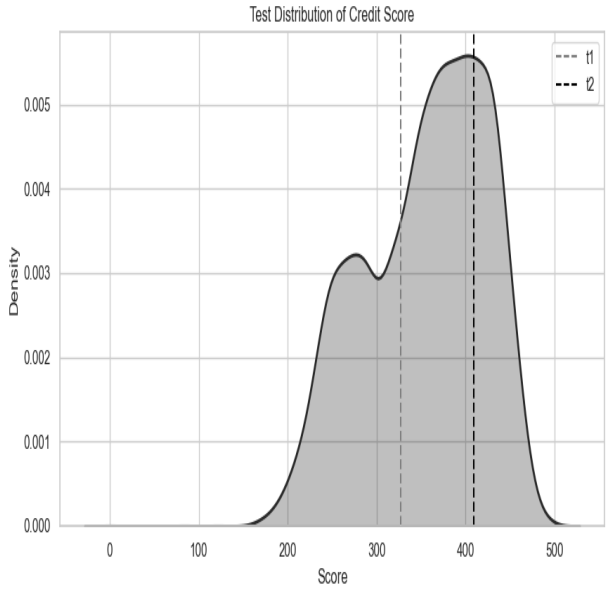

Figure VI. b : Distribution of Credit Score - Test

The distribution of the credit scores provides an overview of how the scoring model allocates observations across the risk spectrum. In both the training and test datasets, most observations concentrate within the central score range, corresponding to the Standard risk category. This pattern is common in credit scoring applications, where many borrowers typically exhibit moderate credit profiles rather than extreme levels of risk (Thomas, Crook & Edelman, 2017).

The vertical dashed lines represent the score thresholds previously defined in the model, $t_1$and t_2, which partition the continuous score into the three credit risk categories. These thresholds act as decision boundaries that translate the score into discrete classifications. The concentration of observations between the thresholds indicates that a large portion of borrowers falls within the intermediate risk region, a characteristic frequently observed in consumer credit datasets (Hand & Henley, 1997).

6.2.2 Score Distribution per Class (Combined)

Training Set

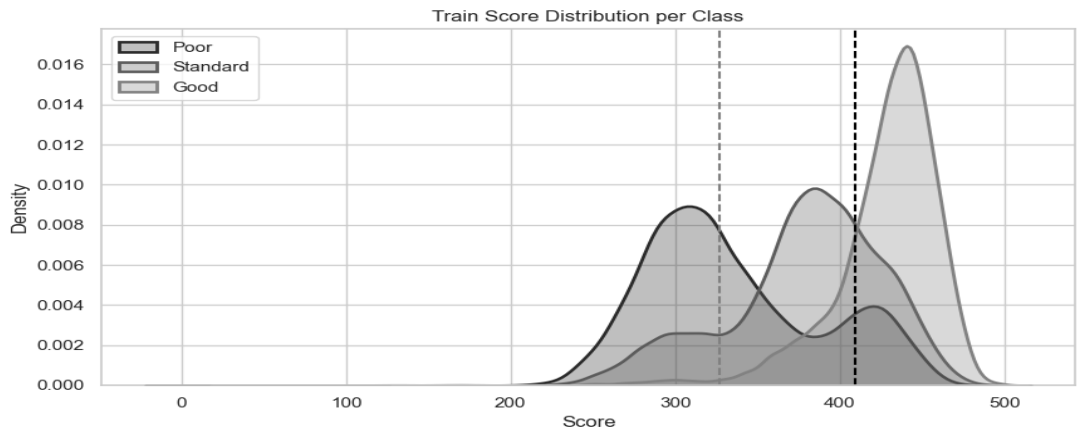

Figure VII. a : Score Distribution per Class (Combined) - Train

Test Set

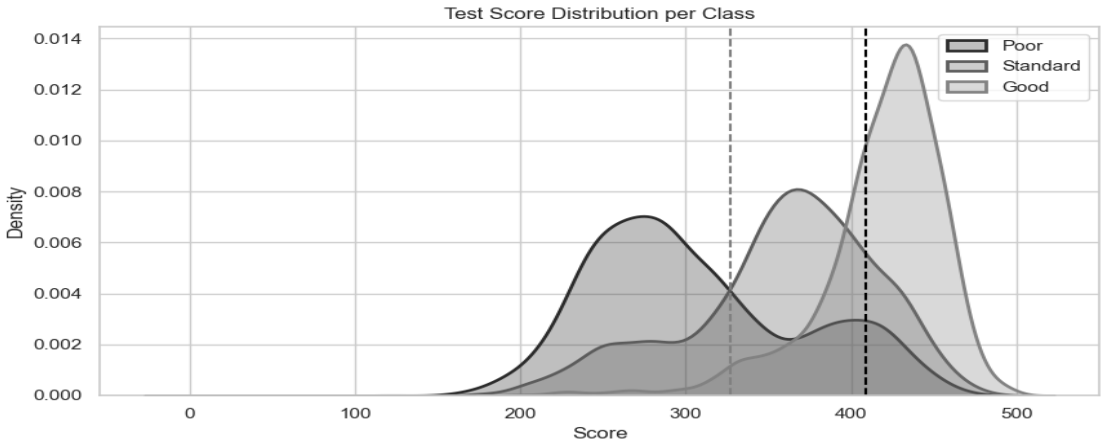

Figure VII. b : Score Distribution per Class (Combined) - Test

The combined score distributions show how the model separates the three credit risk categories along the score axis. In both the training and test datasets, the Poor class is concentrated in the lower score range, while the Good class appears clearly shifted toward higher scores. The Standard category occupies the intermediate region between both groups. Although some overlap can still be observed between the Poor and Standard distributions, the general ordering of the classes remains consistent across both datasets. As discussed previously, the thresholds t_1 and t_2 define the boundaries used to partition this continuous score into the three credit risk categories.

6.2.3 Score Distribution per Class

Training Set

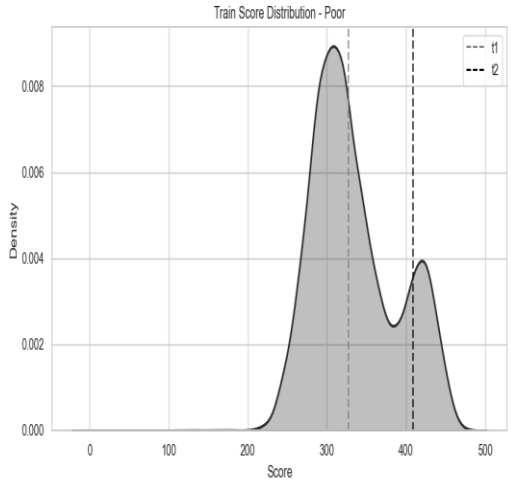

Figure VIII. a : Score Distribution per Class - Train

The score distribution for the Poor class is primarily concentrated in the lower score range, reflecting borrowers with weaker financial profiles. Most observations fall below or near the first threshold t_1, which is consistent with the expected behavior of high-risk borrowers within a credit scoring framework. Some observations extend toward the intermediate region, indicating that certain borrowers classified as Poor may still share characteristics with moderate-risk profiles.

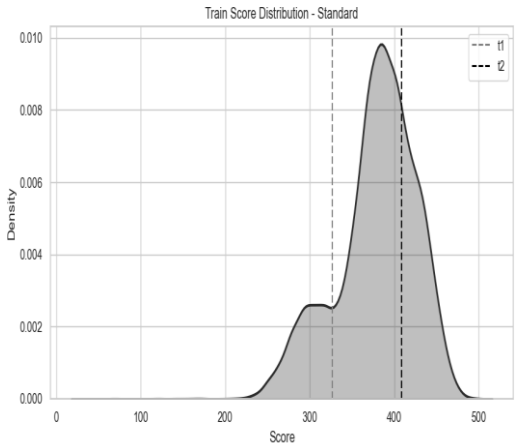

Figure VIII. b : Score Distribution per Class - Train

The Standard class shows a distribution centered around the intermediate score region, located between the thresholds t_1 and t_2. This pattern reflects the transitional nature of this category, which represents borrowers with moderate credit risk. The wider spread of the distribution suggests that this group contains a more heterogeneous set of financial profiles, which explains why some overlap can occur with the neighboring Poor and Good classes.

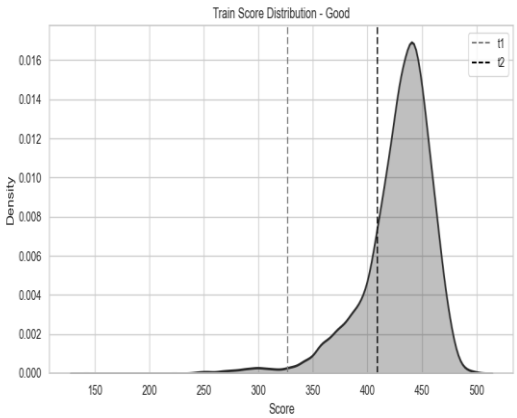

Figure VIII. c : Score Distribution per Class - Train

The distribution for the Good class is clearly shifted toward higher score values, with most observations located above the threshold t_2. This indicates that borrowers classified as Good consistently exhibit stronger financial characteristics associated with lower credit risk. Compared to the other classes, the distribution appears more concentrated, suggesting that low-risk borrowers are more clearly separated by the scoring function.

Test Set

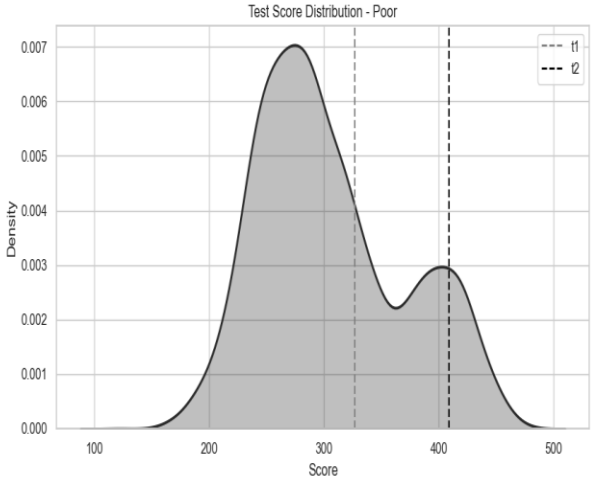

Figure IX. a : Score Distribution per Class - Test

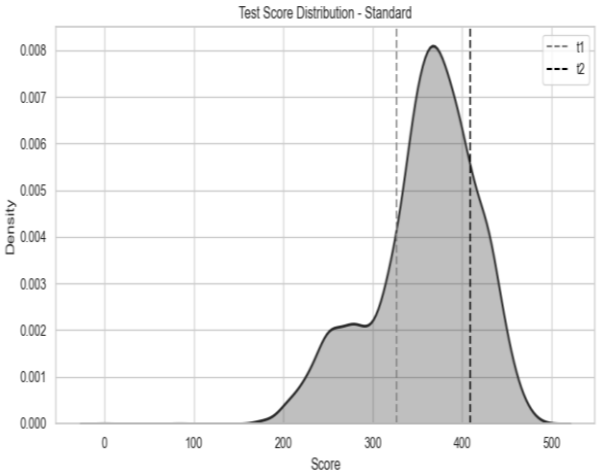

Figure IX. b : Score Distribution per Class - Test

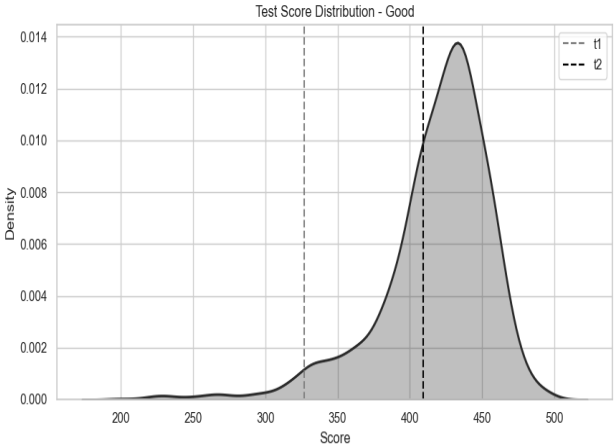

Figure IX. c : Score Distribution per Class - Test

The score distributions for the test dataset exhibit a pattern consistent with the one observed in the training set, indicating that the model maintains a similar ordering of credit risk profiles when applied to unseen data. The Poor class remains concentrated in the lower score region, although the distribution appears slightly wider, suggesting a greater variability in the risk profiles of these borrowers. The Standard class continues to occupy the intermediate score range and shows a relatively stable concentration around the decision thresholds. Finally, the Good class remains clearly shifted toward higher score values, preserving a strong separation from the lower-risk categories. Overall, while a slightly larger overlap between Poor and Standard can be observed compared to the training set, the general structure of the score ranking remains consistent, suggesting that the model generalizes reasonably well to new observations.

6.2.4 Real vs Predicted Score Distributions per Class

Training Set

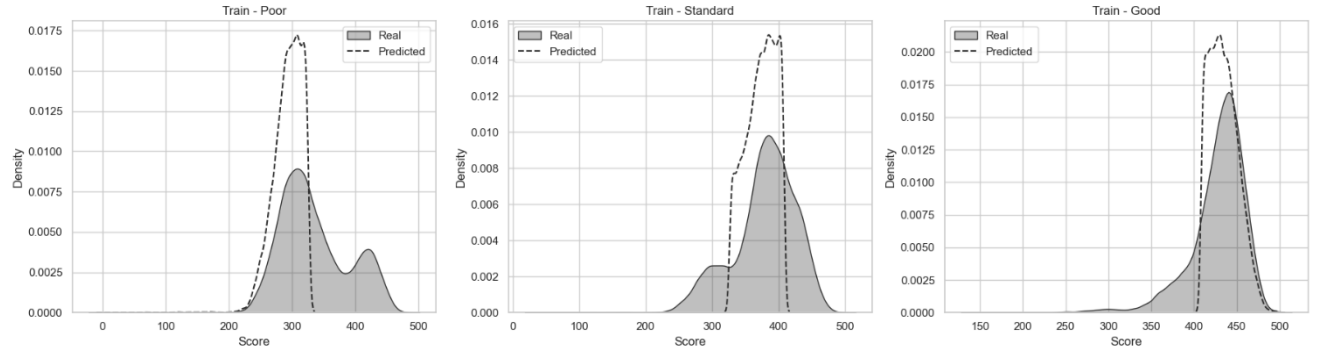

Figure X. a : Real vs Predicted Score Distributions per Class - Train

Test Set

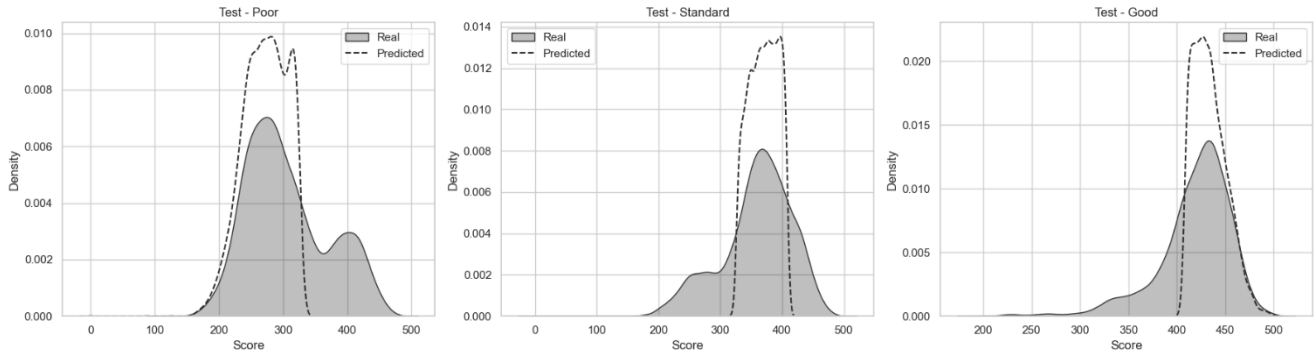

Figure X. b : Real vs Predicted Score Distributions per Class - Test

The comparison between the real and predicted score distributions provides additional insight into how well the scoring model captures the underlying structure of the data. In both the training and test datasets, the predicted distributions generally follow the same shape and ordering as the real distributions across the three credit risk categories. The Poor class remains concentrated in the lower score region, the Standard class occupies the intermediate range, and the Good class is clearly shifted toward higher score values. Although the predicted curves appear slightly more concentrated than the real distributions, the overall alignment between both curves suggests that the scoring model preserves the relative ranking of borrower risk profiles. Minor deviations between the curves reflect the natural uncertainty present in classification tasks and are consistent with the overlap previously observed between neighboring classes.

Assumptions

For the development of this project, which involved first building a multiclass logistic model and subsequently translating it into a traditional scorecard model, the following assumptions were made:

From a statistical standpoint, the logistic regression model was developed under the assumption of linearity between the selected variables and the log-odds of an individual belonging to a specific risk category. In other words, it is presupposed that the impact exhibited by each variable is additive and independent. This was addressed during the correlation matrix analysis prior to selecting the final variables for the model. Likewise, the independence of the observations and/or records in the dataset used for this exercise was assumed, implying that the status of one client does not causally influence the classification of another within the same sample.

Regarding the economic and financial domain, the model was developed under the premise that a client's present behavior or status is a valid and predictive argument for their future financial solvency; that is, it is assumed that their environmental conditions remain stable over time. Unforeseen situations that could affect the client, such as periods of macroeconomic recession or growth, are not considered. Consequently, the client's final credit score is isolated.

Finally, concerning borrower behavior, it is assumed that the borrower is rational and will act appropriately regarding their obligations. The model presented to identify liquidity issues captures the presence of patterns such as periodic delays in credit payments, as well as the reliance on multiple credit cards and the habit of making only the minimum monthly payment. In this sense, the model is strict and, similar to the previous point, does not consider events external to the individual. This assumption is supported by the argument that a progressive and constant deterioration in payment discipline is one of the most reliable indicators for anticipating default (Board of Governors of the Federal Reserve System, 2007; Thomas, 2000).

Results and Discussion

As a result of the exercise conducted, the 62.3% accuracy performance obtained can be interpreted as a good starting point, considering it is a simple model and considering the assumed premises and the random nature that is intrinsic to the financial behavior of each individual. This is unlike a sophisticated machine learning model, which, although it might be more accurate, seeks to predict a client's solvency rather than merely classify it, leading to results that are a priori difficult to interpret or justify financially. Therefore, this result, developed from a traditional model, despite not showing perfect accuracy, does seek to provide a more realistic explanation of the individual behavior of a sample of clients.

That being said, the achieved accuracy demonstrates the capacity to correctly discriminate between classes and to avoid falling into biases or common overfitting problems present in machine learning models. Another piece of evidence supporting the utility of this model is shown in the confusion matrix (Figure XV, a. b.), where Type I error is considerably limited to a scenario where a "Poor" class client would be classified as "Standard" or "Good". This is important because, in such a scenario, a loss would very likely be incurred, as credit would be erroneously granted to a candidate with a high probability of default.

While the results of this project are good, there is still room for improvement, such as the optimization of the thresholds that were set for the classification cut-off among the given scores, which will be discussed later. Being a multiclass classification problem, it was expected that there would be an overlap that would cause noise in the classification for a traditional linear model. In this regard, it would be interesting to approach this exercise using tools or methods that help capture non-linear relationships with the target variable, *"Credit_Score",* helping to reduce the rigidity of the current model's design.

Limitations

Despite the predictive performance of the developed model, which achieves an accuracy of 62.3% on the test set and effectively restricts Type I error, its practical application is subject to various theoretical and methodological restrictions inherent to its design. Throughout the development of this project, the following limitations were encountered.

First, as mentioned in the discussion, the model operates under the assumption of a static economic environment. As assumed throughout the development of this exercise, the borrower's behavior is considered continuous and valid. This initial approach omits and renders the model incapable of adapting to macroeconomic fluctuations; in the event of an extraordinary occurrence, it might be necessary to completely reformulate the model to evaluate different financial aspects of an individual.

Second, being a linear model, it sacrifices the ability to capture non-linear interactions that could provide considerably significant classifying power when incorporating new variables into the study and the construction of the credit model. This was detected during the feature analysis phase, where an experimental model using XGBoost (which was ultimately discarded) achieved an accuracy score of over 80%, successfully identifying patterns that were initially hidden from plain sight.

Another limitation of this exercise is the rigidity with which the model's classification thresholds (t_1 y t_2). were established. Within this project, the limits were set arbitrarily after analyzing the distribution of classified clients at the position that best fitted the intersection between classes (Figure XVII, a, b). However, in a real-world environment, the location of an optimal cutoff point between classes can be dynamic and adjusted to the needs or requirements of a credit institution. This implies that our limits could be adequate or subject to improvement depending on the institution's objectives.

Lastly, the reliability and efficiency of the presented model inherently depend on the quality of the information processed. With a dataset like the one in this exercise, which contained missing values and data errors, the manual correction and cleaning of the data could have limited the model's final performance.

Architecture

10.1 Project Architecture

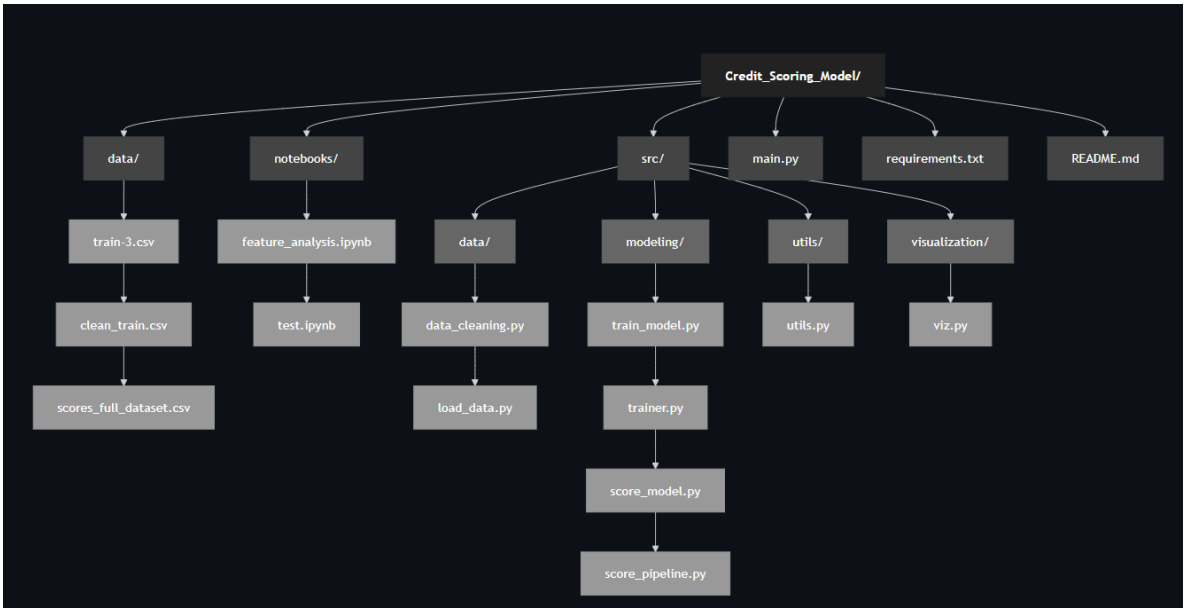

Figure XI. a : Project Architecture of the Credit Scoring Model

10.2 Functional Architecture

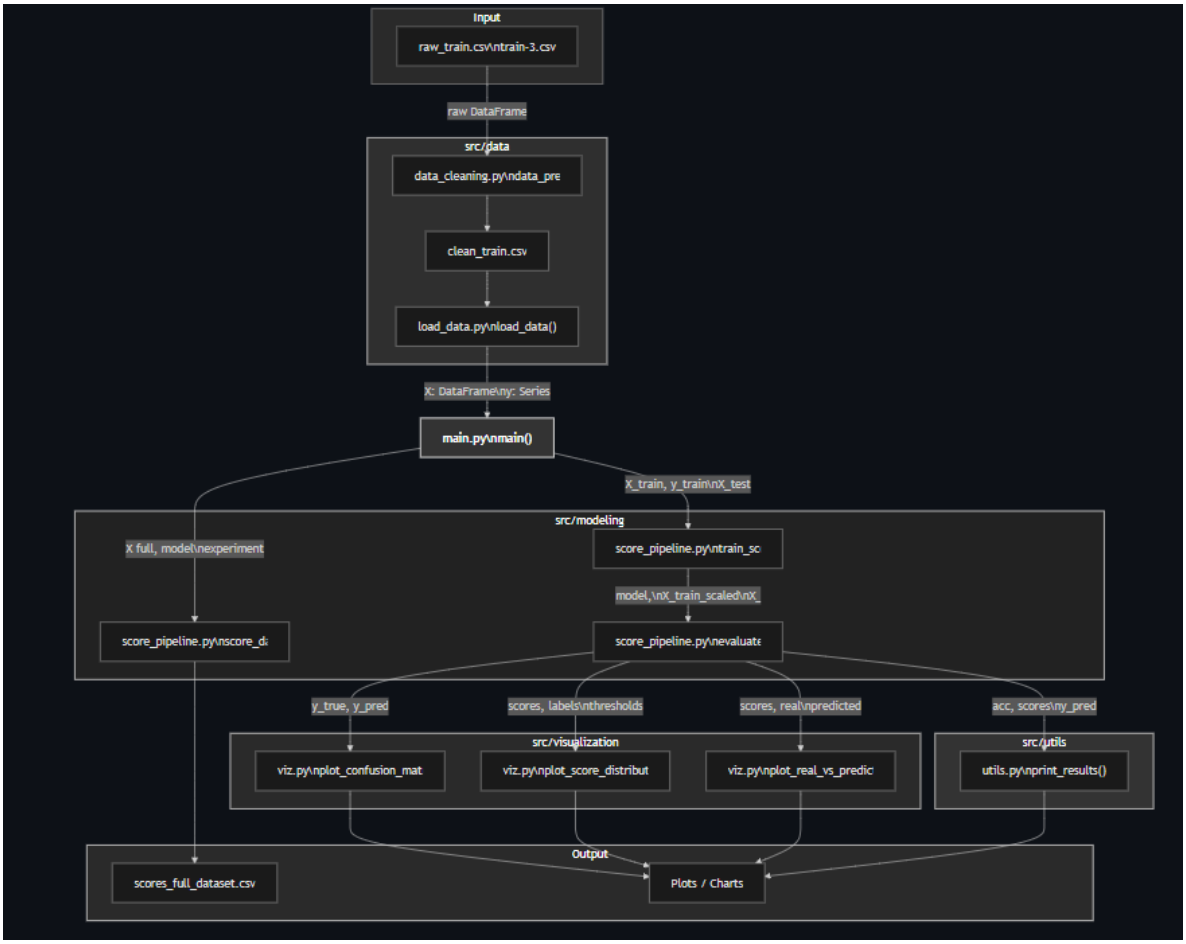

Figure XI . b : Project Functional Architecture of the Credit Scoring Model

10.3 Object-Oriented Architecture

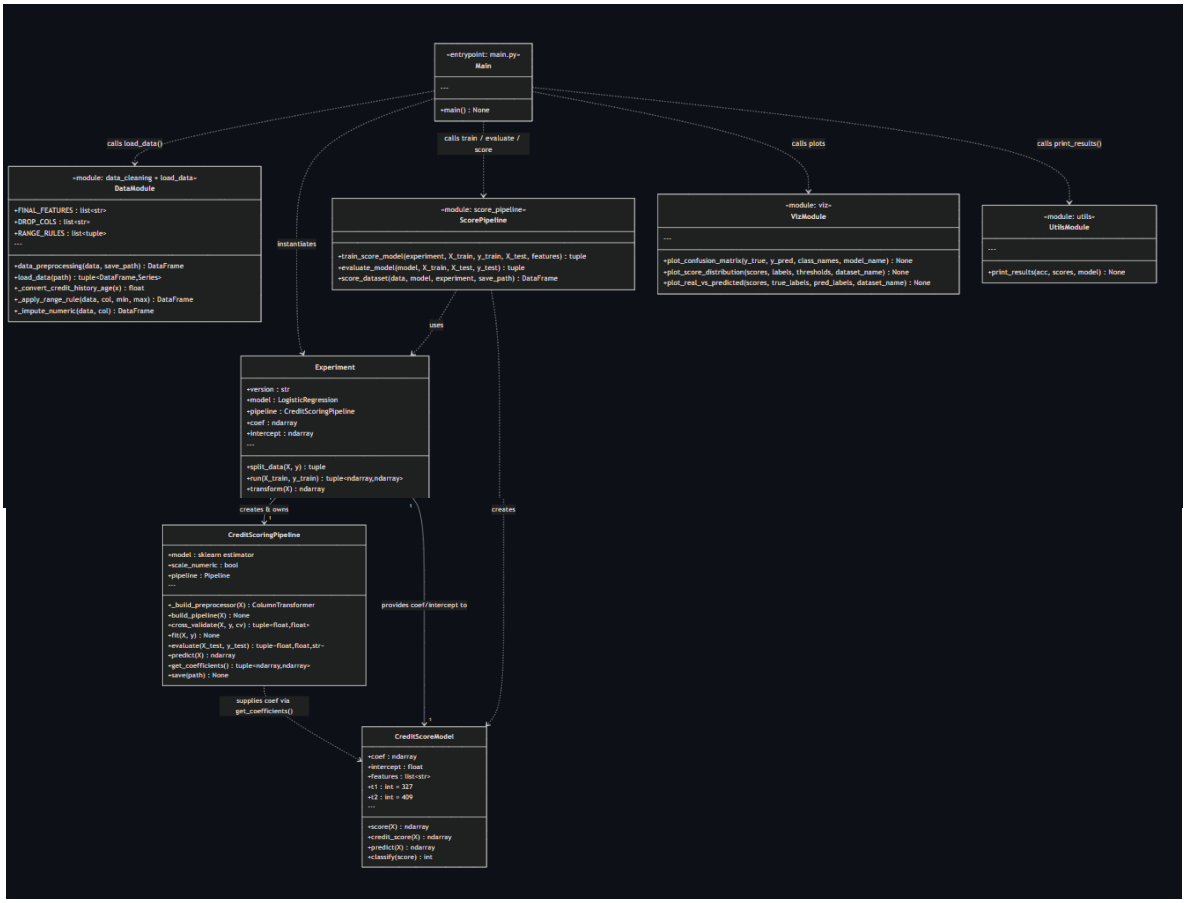

Figure XI. c : Project OOP Architecture of the Credit Scoring Model

Conclusions

The objective of building a traditional scorecard model goes beyond achieving an algorithm capable of discriminating between classes of a data sample; as developed throughout this project, the true objective of this shows as a tool that helps to synthesize human behavior and financial history into a simple and accessible risk metric. From the perspective of an institution, this model demonstrated considerable performance by minimizing critical classification errors, such as approving credit to clients who do not have the capacity to pay. The latter is important because it not only protects the liquidity and structure of a credit institution but also fulfills a social function and/or commitment by protecting individuals from falling into over indebtedness when they are not aware of or cannot control their own financial health.

Returning to the model, beyond the theoretical part, the implementation of a project like this in a practical context served to show real challenges that go hand in hand with its creation. In our case, the first challenge we faced was due to the very noise that came internally in the dataset used. With our tests and during the development of the project, we realized that the success of a credit model depends to a large extent on the quality of the data obtained, which is why we emphasize the importance that models should adapt and continue to evolve according to the context of the environment by incorporating new variables that were not previously considered. Through this project, we worked with the best variables found, standardized to a scale of 300 to 850 imitating the FICO range to facilitate the model's interpretability; however, as mentioned in the limitations, this demonstrated to us that establishing classification thresholds depends more on an institution's decision or strategy to protect itself from risk than on a simple statistical calculation.

Overcoming these types of obstacles in the development of a traditional credit model left us with a new vision regarding its real utility, which is that during our training as financial engineers, we have constantly worked with ML algorithms prioritizing the mathematical precision of the results. However, even though these methods may be more exact, in the end, a simple linear model can be more attractive due to its indispensable capacity to be interpreted in order to economically and financially justify the general impact of each point in the model's

architecture for the final score. In this sense, we now understand the importance of a model's explainability as an operational requirement in credit banking; since, for this and other reasons, traditional models like those proposed by Altman financially grounded, transparent, and efficiently maintain their relevance today and continue to be used by credit institutions.

Bibliography

- *Dataset — Credit score classification*. (2022, June 22). https://www.kaggle.com/datasets/parisrohan/credit-score-classification/data
- pandas documentation — pandas 3.0.1 documentation. (n.d.). https://pandas.pydata.org/docs/
- NumPy documentation — NumPy v2.4 Manual. (n.d.). https://numpy.org/doc/2.4/
- seaborn: statistical data visualization — seaborn 0.13.2 documentation. (n.d.). https://seaborn.pydata.org/
- SHAP documentation — SHAP latest documentation. (n.d.). https://shap.readthedocs.io/en/latest/
- API Reference — SHAP latest documentation. (n.d.). https://shap.readthedocs.io/en/latest/api.html#explanation
- scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation. (n.d.). https://scikit-learn.org/stable/index.html
- GradientBoostingClassifier. (n.d.). Scikit-learn. https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html
- Altman, E. I. (2018). Applications of Distress Prediction Models: What Have We Learned After 50 Years from the Z-Score Models? International Journal of Financial Studies, 6(3), 70. https://doi.org/10.3390/ijfs6030070
- World Bank Group. (2019). Credit Scoring Approaches Guidelines. Washington, DC: World Bank. Recuperado de https://thedocs.worldbank.org/en/doc/935891585869698451-0130022020/original/CREDITSCORINGAPPROACHESGUIDELINESFINALWEB.pdf
- Yotsawat, W. (2021). Improved credit scoring model using XGBoost with Bayesian hyper-parameter optimization. International Journal of Electrical and Computer Engineering, 11(6), 5477-5487. https://ijece.iaescore.com/index.php/IJECE/article/view/24469/15274
- Gramegna, A., & Giudici, P. (2021). SHAP and LIME: An Evaluation of Discriminative Power in Credit Risk. *Frontiers in Artificial Intelligence*, 4, 752558. https://doi.org/10.3389/frai.2021.752558
- Consumer Financial Protection Bureau. (2024). *Consumer credit reporting data* (Office of Research Working Paper). https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4907035
- Altman, E. I. (2000). *Predicting financial distress of companies: Revisiting the Z-score and ZETA models*. Stern School of Business, New York University.
- Avery, R. B., Bostic, R. W., Calem, P. S., & Canner, G. B. (2003). An overview of consumer data and credit reporting. *Federal Reserve Bulletin*, 89, 47-73. https://www.federalreserve.gov/pubs/bulletin/2003/0203lead.pdf
- Board of Governors of the Federal Reserve System. (2007). *Report to the Congress on Credit Scoring and Its Effects on the Availability and Affordability of Credit*. Washington, DC. https://www.federalreserve.gov/boarddocs/rptcongress/creditscore/creditscore.pdf
- Thomas, L. C. (2000). A survey of credit and behavioural scoring: forecasting financial risk of lending to consumers. *International Journal of Forecasting*, 16(2), 149-172. https://doi.org/10.1016/S0169-2070(00)00034-0
- Kagan, J., & Khartit, K. (2020, December 1). *Risk-based pricing: What it means, how it works*. Investopedia. https://www.investopedia.com/terms/r/riskbased-pricing.asp
- The functional and object-oriented architecture diagrams were produced using Claude (Anthropic, 2026) https://claude.ai
- OpenAI. (2026). *ChatGPT (GPT-5.3)* [Large language model]. https://chat.openai.com/

- Stiglitz, J. E., & Weiss, A. (1981). Credit rationing in markets with imperfect information. *The American Economic Review.* *https://pages.ucsd.edu/~aronatas/project/academic/Stiglitz%20credit.pdf*
- Basel Committee on Banking Supervision. (2005). Studies on the Validation of Internal Rating Systems. Bank for International Settlements. https://www.bis.org/publ/bcbs_wp14.pdf

Appendix

Appendix A – Main Script

main.py

```
from src.data.load_data import load_data
from src.utils.utils import print_results
from src.modeling.trainer import Experiment
from src.modeling.score_pipeline import evaluate_model, score_dataset,
train_score_model
from src.visualization.viz import plot_confusion_matrix,
plot_real_vs_predicted, plot_score_distribution


def main():
    """
    Run the end-to-end credit scoring pipeline.

    The workflow includes:
    - Loading the cleaned dataset.
    - Splitting data into train and test sets.
    - Training a logistic-regression-based scorecard model.
    - Evaluating performance and generating predictions.
    - Visualizing confusion matrices and score distributions.
    - Saving scores for the complete dataset.
    """

    X, y = load_data("data/clean_train.csv")

    experiment = Experiment()

    # == Data Splitting ==

    X_train, X_test, y_train, y_test = experiment.split_data(X, y)

    # == Model Training ==

    model, X_train_scaled, X_test_scaled = train_score_model(
        experiment, X_train, y_train, X_test, X.columns)

    # == Model Evaluation ==
```

```
    acc, scores_train, scores_test, y_pred_train, y_pred_test =
evaluate_model(
        model, X_train_scaled, X_test_scaled, y_test)

    # == Results and Visualization ==

    print_results(acc, model)

    plot_confusion_matrix(y_train, y_pred_train, model_name="Train")
    plot_confusion_matrix(y_test, y_pred_test, model_name="Test")

    plot_score_distribution(scores_train, y_train, [
                            model.t1, model.t2], "Train")
    plot_score_distribution(scores_test, y_test, [model.t1, model.t2],
"Test")

    plot_real_vs_predicted(scores_train, y_train, y_pred_train, "Train")
    plot_real_vs_predicted(scores_test, y_test, y_pred_test, "Test")

    # === Scores for Full Dataset ===
    score_dataset(X.copy(), model, experiment,
                  save_path="data/scores_full_dataset.csv")


if __name__ == "__main__":
    main()
```

Appendix B – Data Processing

src/data/load_data.py

```
import pandas as pd

FINAL_FEATURES = [
    'Outstanding_Debt',
    'Interest_Rate',
    'Delay_from_due_date',
    'Num_Credit_Card',
    'Changed_Credit_Limit',
    'Total_EMI_per_month',
    'Credit_Mix_Standard',
    'Credit_Mix_Good',
```

```python
    'Payment_of_Min_Amount_Yes'
]


def load_data(path: str) -> tuple[pd.DataFrame, pd.Series]:
    """
    Load and prepare the dataset for modeling.

    This function performs the following steps:
        1. Reads the dataset from the specified path.
        2. Maps the target variable `Credit_Score` into numeric labels.
        3. Separates features (X) and target (y).
        4. Converts categorical variables into dummy variables.
        5. Selects the final set of features used for modeling.

    Parameters
    ----------
    path : str
        File path to the CSV dataset.

    Returns
    -------
    X : pd.DataFrame
        Feature matrix containing the selected variables.
    y : pd.Series
        Target variable encoded as:
            0 -> Poor
            1 -> Standard
            2 -> Good
    """

    df = pd.read_csv(path)

    mapping = {
        "Poor": 0,
        "Standard": 1,
        "Good": 2
    }

    y = df["Credit_Score"].map(mapping)

    X = df.drop("Credit_Score", axis=1)

    X = pd.get_dummies(X, drop_first=True).astype(float)
```

```
    X = X[FINAL_FEATURES]

    return X, y
```

src/data/data_cleaning.py

```
import re
import numpy as np
import pandas as pd

DROP_COLS = ["ID", "Customer_ID", "Month", "Name", "SSN"]

FILL_MISSING_STR = [
    "Credit_Mix",
    "Occupation",
    "Type_of_Loan",
    "Payment_Behaviour",
]

STRIP_NOISE_COLS = [
    "Age",
    "Annual_Income",
    "Num_of_Loan",
    "Num_of_Delayed_Payment",
    "Changed_Credit_Limit",
    "Outstanding_Debt",
    "Amount_invested_monthly",
    "Monthly_Balance",
    "Num_Credit_Inquiries",
    "Total_EMI_per_month",
    "Credit_Utilization_Ratio",
]

RANGE_RULES = [
    ("Age", 18, 100),
    ("Num_of_Loan", 0, 50),
    ("Num_Bank_Accounts", 0, 20),
    ("Num_Credit_Card", 0, 50),
    ("Interest_Rate", 0, 100),
    ("Credit_History_Age", 0.01, None),
]
```

```python
def _convert_credit_history_age(x) -> float:
    """
    Convert a credit history string into decimal years.

    Example
    -------
    '22 Years and 3 Months' → 22.25

    Parameters
    ----------
    x : str | NaN
        Original credit history value.

    Returns
    -------
    float
        Credit history expressed in years. Returns NaN if the input is missing.
    """

    if pd.isna(x):
        return np.nan

    years = re.search(r"(\d+)\s+Years", str(x))
    months = re.search(r"(\d+)\s+Months", str(x))

    y = int(years.group(1)) if years else 0
    m = int(months.group(1)) if months else 0

    return round(y + m / 12, 4)


def _apply_range_rule(data: pd.DataFrame, col: str, min_val: float | None,
max_val: float | None) -> pd.DataFrame:
    """
    Enforce logical bounds on numeric variables.

    Values outside the allowed range are replaced with NaN and an
    indicator column `<column>_invalid` is created.

    Parameters
    ----------
    data : pd.DataFrame
        Input dataset.
```

```
    col : str
        Column to validate.
    min_val : float | None
        Minimum allowed value.
    max_val : float | None
        Maximum allowed value.

    Returns
    -------
    pd.DataFrame
        Updated dataset with invalid values handled.
    """

    mask = pd.Series(False, index=data.index)

    if min_val is not None:
        mask |= data[col] < min_val
    if max_val is not None:
        mask |= data[col] > max_val

    if mask.any():
        data[f"{col}_invalid"] = mask.astype(int)
        data.loc[mask, col] = np.nan

    return data


def _impute_numeric(data: pd.DataFrame, col: str) -> pd.DataFrame:
    """
    Impute missing values in numeric columns using the median.

    Also creates a `<column>_missing` indicator when missing values exist.

    Parameters
    ----------
    data : pd.DataFrame
        Dataset containing the column.
    col : str
        Numeric column to impute.

    Returns
    -------
    pd.DataFrame
        Dataset with imputed values.
```

```python
    """

    if data[col].isna().any():
        data[f"{col}_missing"] = data[col].isna().astype(int)
        median_val = data[col].median()
        data[col] = data[col].fillna(median_val)

    return data


def data_preprocessing(data: pd.DataFrame, save_path: str | None = None) ->
pd.DataFrame:
    """
    Execute the full preprocessing pipeline for the Credit Score dataset.

    Processing Steps
    ----------------
    1. Remove identification and PII columns.
    2. Clean corrupted string patterns and noise.
    3. Convert noisy numeric fields to numeric values.
    4. Transform `Credit_History_Age` into decimal years.
    5. Fill missing categorical values with `"Missing"`.
    6. Apply logical range validation rules.
    7. Impute remaining numeric values using the median.
    8. Normalize categorical inconsistencies.

    Parameters
    ----------
    data : pd.DataFrame
        Raw input dataset.
    save_path : str | None, optional
        If provided, the cleaned dataset is saved to this path.

    Returns
    -------
    pd.DataFrame
        Cleaned dataset ready for modeling.
    """

    data = data.copy()

    # Drop identification columns
    data.drop(columns=DROP_COLS, inplace=True, errors="ignore")
```

```python
    # Remove global string noise
    data = data.replace({r"_+": "", r"!@9#%8": ""}, regex=True)
    data = data.replace("", np.nan)

    # Convert noisy numeric columns
    for col in STRIP_NOISE_COLS:
        if col in data.columns:
            data[col] = (
                data[col]
                .astype(str)
                .str.replace(r"[^\d.]", "", regex=True)
                .pipe(pd.to_numeric, errors="coerce")
            )

    # Convert Credit_History_Age
    if "Credit_History_Age" in data.columns:
        data["Credit_History_Age"] = data["Credit_History_Age"].apply(
            _convert_credit_history_age
        )

    # Impute categorical columns
    for col in FILL_MISSING_STR:
        if col in data.columns:
            data[col] = data[col].fillna("Missing")

    # Apply logical range rules
    for col, lo, hi in RANGE_RULES:
        if col in data.columns:
            data = _apply_range_rule(data, col, lo, hi)

    # Impute numeric columns
    numeric_cols = data.select_dtypes(include=["int64", "float64"]).columns

    for col in numeric_cols:
        if not col.endswith(("_invalid", "_missing")):
            data = _impute_numeric(data, col)

    # Normalize categorical inconsistencies
    if "Payment_of_Min_Amount" in data.columns:
        data["Payment_of_Min_Amount"] = (
            data["Payment_of_Min_Amount"]
            .astype(str)
            .str.replace("NM", "No", regex=False)
        )
```

```
    if save_path is not None:
        data.to_csv(save_path, index=False)

    return data
```

Appendix C – Model Training

src/modeling/train_model.py

```
import joblib
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score, classification_report,
roc_auc_score


class CreditScoringPipeline:
    """
    End-to-end pipeline for training and evaluating a credit scoring model.

    This class encapsulates preprocessing and model training using a
    scikit-learn Pipeline. It provides utilities for cross-validation,
    evaluation, prediction, and coefficient extraction from the trained
model.
    """

    def __init__(self, model, scale_numeric: bool = True) -> None:
        """
        Initialize the pipeline.

        Parameters
        ----------
        model : sklearn estimator
            Machine learning model used for training (e.g.,
LogisticRegression).
        scale_numeric : bool, optional
            Whether to apply StandardScaler to numeric features.
```

```python
        """
        self.model = model
        self.scale_numeric = scale_numeric
        self.pipeline = None

    def _build_preprocessor(self, X: pd.DataFrame) -> ColumnTransformer:
        """
        Create the preprocessing component of the pipeline.

        Parameters
        ----------
        X : pd.DataFrame
            Feature dataset used to determine numeric columns.

        Returns
        -------
        ColumnTransformer
            Preprocessing transformer that scales numeric features.
        """
        numeric_cols = X.columns

        numeric_transformer = (
            StandardScaler() if self.scale_numeric else "passthrough"
        )

        preprocessor = ColumnTransformer(
            transformers=[
                ("num", numeric_transformer, numeric_cols)
            ]
        )

        return preprocessor

    def build_pipeline(self, X: pd.DataFrame) -> None:
        """
        Build the full training pipeline.

        The pipeline consists of:
            preprocessing → model training

        Parameters
        ----------
        X : pd.DataFrame
            Feature dataset used to configure preprocessing.
```

```
        """
        if self.pipeline is not None:
            return

        preprocessor = self._build_preprocessor(X)

        self.pipeline = Pipeline([
            ("preprocessor", preprocessor),
            ("model", self.model)
        ])

    def cross_validate(self, X: pd.DataFrame, y: pd.Series | np.ndarray, cv:
int = 5) -> tuple[float, float]:
        """
        Perform stratified cross-validation on the pipeline.

        Parameters
        ----------
        X : pd.DataFrame
            Feature dataset.
        y : pd.Series or np.ndarray
            Target variable.
        cv : int, optional
            Number of cross-validation folds.

        Returns
        -------
        tuple
            Mean and standard deviation of the F1 macro score.
        """
        self.build_pipeline(X)

        skf = StratifiedKFold(
            n_splits=cv,
            shuffle=True,
            random_state=42
        )

        scores = cross_val_score(
            self.pipeline,
            X,
            y,
            cv=skf,
            scoring="f1_macro",
```

```
            n_jobs=-1
        )

        return np.mean(scores), np.std(scores)

    def fit(self, X: pd.DataFrame, y: pd.Series | np.ndarray) -> None:
        """
        Train the pipeline on the provided dataset.

        Parameters
        ----------
        X : pd.DataFrame
            Feature dataset.
        y : pd.Series or np.ndarray
            Target variable.
        """
        self.build_pipeline(X)
        self.pipeline.fit(X, y)

    def evaluate(self, X_test: pd.DataFrame, y_test: pd.Series | np.ndarray)
-> tuple[float, float, str]:
        """
        Evaluate model performance on the test dataset.

        Parameters
        ----------
        X_test : pd.DataFrame
            Test feature dataset.
        y_test : pd.Series or np.ndarray
            True labels for the test dataset.

        Returns
        -------
        tuple
            accuracy : float
                Classification accuracy.
            auc : float
                Multi-class ROC AUC score.
            report : str
                Full classification report.
        """
        y_pred = self.pipeline.predict(X_test)
        y_proba = self.pipeline.predict_proba(X_test)
```

```python
        acc = accuracy_score(y_test, y_pred)

        auc = roc_auc_score(
            y_test,
            y_proba,
            multi_class="ovr",
            average="macro"
        )

        report = classification_report(y_test, y_pred)

        return acc, auc, report

    def predict(self, X: pd.DataFrame) -> np.ndarray:
        """
        Generate predictions using the trained pipeline.

        Parameters
        ----------
        X : pd.DataFrame
            Feature dataset.

        Returns
        -------
        np.ndarray
            Predicted class labels.
        """
        return self.pipeline.predict(X)

    def get_coefficients(self) -> tuple[np.ndarray, np.ndarray]:
        """
        Extract model coefficients and intercepts.

        Returns
        -------
        tuple
            coef : np.ndarray
                Model coefficients.
            intercept : np.ndarray
                Model intercept values.
        """
        model = self.pipeline.named_steps["model"]

        return model.coef_, model.intercept_
```

```python
    def save(self, path: str) -> None:
        """
        Save the trained pipeline to disk.

        Parameters
        ----------
        path : str
            File path where the pipeline will be stored.
        """
        joblib.dump(self.pipeline, path)
```

src/modeling/trainer.py

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from src.modeling.train_model import CreditScoringPipeline


class Experiment:
    """
    Manage a credit scoring training experiment.

    This class handles dataset splitting, model training through a
    preprocessing pipeline, and storage of learned model parameters.
    """

    def __init__(self, version: str = "v1") -> None:
        """
        Initialize experiment settings.

        Parameters
        ----------
        version : str, optional
            Version identifier for the experiment (default: "v1").
        """
        self.version = version

        self.model = LogisticRegression(
            max_iter=10000,
            class_weight="balanced",
```

```
            random_state=42
        )

        self.pipeline = None
        self.coef = None
        self.intercept = None

    def split_data(self, X: pd.DataFrame, y: pd.Series | np.ndarray) ->
tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
        """
        Split the dataset into training and testing sets.

        Stratified sampling is used to preserve the class distribution.

        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix.
        y : pd.Series or np.ndarray
            Target labels.

        Returns
        -------
        tuple
            X_train, X_test, y_train, y_test
        """
        X_train, X_test, y_train, y_test = train_test_split(
            X,
            y,
            test_size=0.2,
            stratify=y,
            random_state=42
        )

        return X_train, X_test, y_train, y_test

    def run(self, X_train: pd.DataFrame, y_train: pd.Series | np.ndarray) ->
tuple[np.ndarray, np.ndarray]:
        """
        Train the credit scoring pipeline.

        The method fits the preprocessing pipeline and logistic regression
        model, then extracts the learned coefficients.
```

```
        Parameters
        ----------
        X_train : pd.DataFrame
            Training feature matrix.
        y_train : pd.Series or np.ndarray
            Training target labels.

        Returns
        -------
        tuple
            coef : np.ndarray
                Model coefficients.
            intercept : np.ndarray
                Model intercepts.
        """
        pipeline = CreditScoringPipeline(
            self.model,
            scale_numeric=True
        )

        pipeline.fit(X_train, y_train)

        coef, intercept = pipeline.get_coefficients()

        self.pipeline = pipeline
        self.coef = coef
        self.intercept = intercept

        return coef, intercept

    def transform(self, X: pd.DataFrame | np.ndarray) -> np.ndarray:
        """
        Transform features using the fitted preprocessing pipeline.

        Parameters
        ----------
        X : pd.DataFrame or array-like
            Input features.

        Returns
        -------
        np.ndarray
            Transformed feature matrix.
        """
```

```
        return
self.pipeline.pipeline.named_steps["preprocessor"].transform(X)
```

Appendix D – Credit Scoring Pipeline

src/modeling/score_pipeline.py

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score
from src.modeling.trainer import Experiment
from src.modeling.score_model import CreditScoreModel


def train_score_model(experiment: Experiment, X_train: pd.DataFrame,
y_train: pd.Series | np.ndarray, X_test: pd.DataFrame, features: list[str])
-> tuple[CreditScoreModel, np.ndarray, np.ndarray]:
    """
    Train the credit scoring model and prepare transformed datasets.

    This function executes the experiment pipeline to train a logistic
    regression model and extracts coefficients to build a linear
    credit scoring model.

    The score is constructed as the difference between the coefficients
    of the "Good" and "Poor" classes:

        score = logit_good - logit_poor

    The trained preprocessing pipeline is then applied to both the
    training and test datasets.

    Parameters
    ----------
    experiment : Experiment
        Experiment object responsible for fitting the preprocessing
        pipeline and the logistic regression model.
    X_train : pd.DataFrame
        Training feature dataset.
    y_train : pd.Series or np.ndarray
        Training target labels.
    X_test : pd.DataFrame
```

```
        Test feature dataset.
    features : list
        List of feature names used by the model.

    Returns
    -------
    model : CreditScoreModel
        Trained credit scoring model.
    X_train_scaled : np.ndarray
        Transformed training features after preprocessing.
    X_test_scaled : np.ndarray
        Transformed test features after preprocessing.
    """

    coef, intercept = experiment.run(X_train, y_train)

    coef_good = coef[2]
    coef_poor = coef[0]

    coef_score = coef_good - coef_poor
    intercept_score = intercept[2] - intercept[0]

    X_train_scaled = experiment.transform(X_train)
    X_test_scaled = experiment.transform(X_test)

    model = CreditScoreModel(
        coef=coef_score,
        intercept=intercept_score,
        features=features
    )

    return model, X_train_scaled, X_test_scaled


def evaluate_model(model: CreditScoreModel, X_train: np.ndarray, X_test:
np.ndarray, y_test: pd.Series | np.ndarray) -> tuple[float, np.ndarray,
np.ndarray, np.ndarray, np.ndarray]:
    """
    Evaluate the credit scoring model's performance.

    This function computes the credit scores for both the training and
    test datasets, generates predictions, and calculates the classification
    accuracy on the test set.
```

```
    Parameters
    ----------
    model : CreditScoreModel
        Trained credit scoring model.
    X_train : np.ndarray
        Transformed training features.
    X_test : np.ndarray
        Transformed test features.
    y_test : pd.Series or np.ndarray
        True labels for the test dataset.

    Returns
    -------
    acc : float
        Classification accuracy on the test set.
    scores_train : np.ndarray
        Credit scores for the training dataset.
    scores_test : np.ndarray
        Credit scores for the test dataset.
    y_pred_train : np.ndarray
        Predicted class labels for the training dataset.
    y_pred_test : np.ndarray
        Predicted class labels for the test dataset.
    """

    scores_train = model.credit_score(X_train)
    scores_test = model.credit_score(X_test)

    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred_test)

    return acc, scores_train, scores_test, y_pred_train, y_pred_test


def score_dataset(data: pd.DataFrame, model: CreditScoreModel, experiment:
Experiment, save_path: str = "predicted_scores.csv") -> pd.DataFrame:
    """
    Compute credit scores and predicted categories for a dataset.

    Parameters
    ----------
    data : pd.DataFrame
```

```
        Dataset to score.
    model : CreditScoreModel
        Trained credit score model.
    experiment : Experiment
        Experiment object containing the fitted preprocessing pipeline.
    save_path : str, optional
        Path where the scored dataset will be saved.

    Returns
    -------
    pd.DataFrame
        DataFrame containing Score, Classification and Credit_Category.
    """

    scaler = experiment.pipeline.pipeline.named_steps["preprocessor"]

    X = data[model.features]

    X_scaled = scaler.transform(X)

    scores = model.credit_score(X_scaled)

    classification = model.predict(X_scaled)

    category_map = {
        0: "Poor",
        1: "Standard",
        2: "Good"
    }

    result = pd.DataFrame({
        "Score": scores,
        "Classification": classification,
        "Credit_Category": [category_map[c] for c in classification]
    })

    result.to_csv(save_path, index=False)

    return result
```

src/modeling/score_model.py

```
import numpy as np


class CreditScoreModel:
    """
    Credit scoring model based on a logistic regression linear score.

    This class transforms the linear output of a trained logistic regression
    model into a normalized credit score between 300 and 850. It also
provides
    classification of observations into credit categories using predefined
    score thresholds.
    """

    def __init__(self, coef: np.ndarray, intercept: float, features:
list[str]) -> None:
        """
        Initialize the credit score model.

        Args:
            coef (np.ndarray): Coefficients of the logistic regression model
                corresponding to the selected class.
            intercept (float): Intercept term of the logistic regression
model.
            features (list): List of feature names used by the model.
        """
        self.coef = coef
        self.intercept = intercept
        self.features = features
        self.t1 = 327
        self.t2 = 409

    def score(self, X: np.ndarray) -> np.ndarray:
        """
        Compute the linear score (logit) of the model.

        Args:
            X (array-like): Input feature matrix.

        Returns:
            np.ndarray: Linear combination of features and coefficients.
        """
```

```python
        Xv = np.asarray(X)
        z = np.dot(Xv, self.coef) + self.intercept
        return z

    def credit_score(self, X: np.ndarray) -> np.ndarray:
        """
        Convert the model's linear score into a normalized credit score.

        The score is rescaled into the range [300, 850] using min-max
        normalization.

        Args:
            X (array-like): Input feature matrix.

        Returns:
            np.ndarray: Credit scores between 300 and 850.
        """
        z = self.score(X)
        z_min = z.min()
        z_max = z.max()
        score_norm = (z - z_min) / (z_max - z_min)
        score_min = 0
        score_max = 500
        score = score_norm * (score_max - score_min) + score_min
        return score.astype(int)

    def predict(self, X: np.ndarray) -> np.ndarray:
        """
        Predict credit classes based on computed credit scores.

        Args:
            X (array-like): Input feature matrix.

        Returns:
            np.ndarray: Predicted credit class labels.
        """
        scores = self.credit_score(X)
        return np.array([self.classify(s) for s in scores])

    def classify(self, score: int) -> int:
        """
        Assign a credit category based on score thresholds.

        Args:
```

```
            score (int): Credit score value.

        Returns:
            int: Predicted class label
                 0 -> Poor
                 1 -> Standard
                 2 -> Good
        """
        if score < self.t1:
            return 0
        elif score < self.t2:
            return 1
        else:
            return 2
```

Appendix E – Utilities

src/utils/utils.py

```
import numpy as np


def print_results(acc: float | np.ndarray, model: object) -> None:
    """
    Print a summary of model performance and scoring thresholds.

    Parameters
    ----------
    acc : float
        Classification accuracy of the model.
    scores : list or np.ndarray
        Predicted credit scores generated by the model.
    model : object
        Trained scoring model containing the threshold attributes `t1` and
`t2`.
    """
    print("Accuracy:", acc)
    print(f"Thresholds used → t1: {model.t1}, t2: {model.t2}")
```

Appendix F – Visualization

src/visualization/viz.py

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from sklearn.metrics import confusion_matrix


sns.set_theme(style="whitegrid", palette="Greys_r")
plt.rcParams["figure.figsize"] = (12, 6)
plt.rcParams["figure.dpi"] = 100

red_grey = mcolors.LinearSegmentedColormap.from_list(
    "RedGrey",
    ["#bfbfbf96", "#c60f0f"])


def plot_confusion_matrix(y_true: pd.Series | np.ndarray, y_pred:
np.ndarray, class_names: tuple[str, str, str] = ("Poor", "Standard",
"Good"), model_name: str = "Score Model") -> None:
    """
    Plot the confusion matrix of predicted vs true classes.

    Parameters
    ----------
    y_true : pd.Series or np.ndarray
        True class labels.
    y_pred : np.ndarray
        Predicted class labels.
    model_name : str
        Name of the model for the plot title.
    """

    matrix = confusion_matrix(y_true, y_pred)
    matrix_df = pd.DataFrame(matrix, index=class_names, columns=class_names)

    plt.figure(figsize=(8, 6))
    sns.heatmap(matrix_df, annot=True, fmt="d", cmap="Greys", cbar=False)
    plt.title(f"Confusion Matrix - {model_name}")
    plt.ylabel("True Label")
```

```
    plt.xlabel("Predicted Label")
    plt.tight_layout()
    plt.show()


def plot_score_distribution(score_values, true_labels, score_thresholds,
dataset_name="Train"):
    """
    Visualizes the distribution of credit scores generated by the scoring
model.

    This function produces three types of visualizations to analyze how the
    scoring system separates observations across credit risk categories:

    1. Global score distribution across all observations.
    2. Score distributions for all classes plotted together.
    3. Individual score distributions for each credit class.

    The function also displays the decision thresholds (t1 and t2) used to
    convert the continuous credit score into discrete risk categories.

    Parameters
    ----------
    score_values : array-like
        Computed credit score for each observation in the dataset.

    true_labels : array-like
        True class labels associated with each observation. Expected values:
        0 = Poor, 1 = Standard, 2 = Good.

    score_thresholds : list or tuple of length 2
        Threshold values used to partition the score into the three credit
        risk categories. The first threshold corresponds to t1 and the
second
        to t2.

    dataset_name : str, optional
        Name of the dataset being visualized (e.g., "Train" or "Test").
        This value is used in the plot titles.

    Returns
    -------
    None
        The function generates and displays the plots directly.
```

```
    Notes
    -----
    Kernel Density Estimation (KDE) is used to approximate the distribution
    of credit scores. The dashed vertical lines indicate the score
thresholds
    used for risk classification.
    """

    score_df = pd.DataFrame({
        "Score": score_values,
        "True_Class": true_labels
    })

    class_map = {
        0: "Poor",
        1: "Standard",
        2: "Good"
    }

    # -------- GENERAL DISTRIBUTION --------
    plt.figure(figsize=(10, 5))

    sns.kdeplot(
        x=score_df["Score"],
        fill=True,
        alpha=0.3,
        linewidth=2
    )

    plt.axvline(score_thresholds[0], linestyle="--", color="gray",
label="t1")
    plt.axvline(score_thresholds[1], linestyle="--", color="black",
label="t2")

    plt.title(f"{dataset_name} Distribution of Credit Score")
    plt.xlabel("Score")
    plt.ylabel("Density")
    plt.legend()

    plt.show()

    # -------- DISTRIBUTION PER CLASS (LAS 3 JUNTAS) --------
    plt.figure(figsize=(10, 5))
```

```python
for class_id, class_name in class_map.items():

    sns.kdeplot(
        x=score_df.loc[score_df["True_Class"] == class_id, "Score"],
        fill=True,
        alpha=0.3,
        linewidth=2,
        label=class_name
    )

plt.axvline(score_thresholds[0], linestyle="--", color="gray")
plt.axvline(score_thresholds[1], linestyle="--", color="black")

plt.title(f"{dataset_name} Score Distribution per Class")
plt.xlabel("Score")
plt.ylabel("Density")
plt.legend()

plt.show()

# -------- INDIVIDUAL CLASS DISTRIBUTIONS --------
for class_id, class_name in class_map.items():

    plt.figure(figsize=(10, 5))

    sns.kdeplot(
        x=score_df.loc[score_df["True_Class"] == class_id, "Score"],
        fill=True,
        alpha=0.3,
        linewidth=2,
        bw_adjust=1
    )

    plt.axvline(score_thresholds[0],
                linestyle="--", color="gray", label="t1")
    plt.axvline(score_thresholds[1],
                linestyle="--", color="black", label="t2")

    plt.title(f"{dataset_name} Score Distribution - {class_name}")
    plt.xlabel("Score")
    plt.ylabel("Density")
    plt.legend()
```

```
        plt.show()


def plot_real_vs_predicted(score_values: np.ndarray, true_labels: pd.Series
| np.ndarray, predicted_labels: np.ndarray, dataset_name: str = "Train") ->
None:
    """
    Compare real and predicted score distributions for each class.

    Parameters
    ----------
    score_values : np.ndarray
        Credit scores produced by the model.
    true_labels : array-like
        True class labels.
    predicted_labels : np.ndarray
        Predicted class labels.
    dataset_name : str
        Dataset identifier (Train/Test).
    """

    results_df = pd.DataFrame({
        "Score": score_values,
        "Real_Class": true_labels,
        "Predicted_Class": predicted_labels
    })

    label_names = {
        0: "Poor",
        1: "Standard",
        2: "Good"
    }

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for class_id in range(3):

        real_dist = results_df[results_df["Real_Class"] ==
class_id]["Score"]
        pred_dist = results_df[results_df["Predicted_Class"]
                               == class_id]["Score"]

        sns.kdeplot(
            real_dist,
```

```
            ax=axes[class_id],
            label="Real",
            fill=True,
            alpha=0.3
        )

        sns.kdeplot(
            pred_dist,
            ax=axes[class_id],
            label="Predicted",
            linestyle="--"
        )

        axes[class_id].set_title(f"{dataset_name} -
{label_names[class_id]}")
        axes[class_id].set_xlabel("Score")
        axes[class_id].set_ylabel("Density")
        axes[class_id].legend()

    plt.tight_layout()
    plt.show()
```

Appendix G – Project Configuration

requirements.txt

```
alembic==1.18.4
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.12.1
blinker==1.9.0
cachetools==7.0.1
certifi==2026.2.25
cffi==2.0.0
charset-normalizer==3.4.4
click==8.3.1
cloudpickle==3.1.2
colorama==0.4.6
contourpy==1.3.3
cryptography==46.0.5
cycler==0.12.1
databricks-sdk==0.94.0
```

```
docker==7.1.0
fastapi==0.133.1
Flask==3.1.3
flask-cors==6.0.2
fonttools==4.61.1
gitdb==4.0.12
GitPython==3.1.46
google-auth==2.48.0
graphene==3.4.3
graphql-core==3.2.7
graphql-relay==3.2.0
greenlet==3.3.2
h11==0.16.0
huey==2.6.0
idna==3.11
importlib_metadata==8.7.1
itsdangerous==2.2.0
Jinja2==3.1.6
joblib==1.5.3
kiwisolver==1.4.9
lightgbm==4.6.0
llvmlite==0.46.0
Mako==1.3.10
MarkupSafe==3.0.3
matplotlib==3.10.8
mlflow==3.10.0
mlflow-skinny==3.10.0
mlflow-tracing==3.10.0
numba==0.64.0
numpy==2.4.2
opentelemetry-api==1.39.1
opentelemetry-proto==1.39.1
opentelemetry-sdk==1.39.1
opentelemetry-semantic-conventions==0.60b1
packaging==26.0
pandas==2.3.3
pillow==12.1.1
prettytable==3.17.0
protobuf==6.33.5
pyarrow==23.0.1
pyasn1==0.6.2
pyasn1_modules==0.4.2
pycparser==3.0
pydantic==2.12.5
```

```
pydantic_core==2.41.5
pyparsing==3.3.2
python-dateutil==2.9.0.post0
python-dotenv==1.2.1
pytz==2025.2
pywin32==311
PyYAML==6.0.3
requests==2.32.5
rsa==4.9.1
scikit-learn==1.8.0
scipy==1.17.1
shap==0.50.0
six==1.17.0
skops==0.13.0
slicer==0.0.8
smmap==5.0.2
SQLAlchemy==2.0.47
sqlparse==0.5.5
starlette==0.52.1
threadpoolctl==3.6.0
tqdm==4.67.3
typing-inspection==0.4.2
typing_extensions==4.15.0
tzdata==2025.3
urllib3==2.6.3
uvicorn==0.41.0
waitress==3.0.2
wcwidth==0.6.0
Werkzeug==3.1.6
xgboost==3.2.0
zipp==3.23.0
```

.gitignore

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[codz]
*$py.class

# C extensions
*.so

# Distribution / packaging
```

```
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
share/python-wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST

# PyInstaller
#  Usually these files are written by a python script from a template
#  before PyInstaller builds the exe, so as to inject date/other infos into
it.
*.manifest
*.spec

# Installer logs
pip-log.txt
pip-delete-this-directory.txt

# Unit test / coverage reports
htmlcov/
.tox/
.nox/
.coverage
.coverage.*
.cache
nosetests.xml
coverage.xml
*.cover
*.py.cover
.hypothesis/
.pytest_cache/
cover/
```

```
# Translations
*.mo
*.pot

# Django stuff:
*.log
local_settings.py
db.sqlite3
db.sqlite3-journal

# Flask stuff:
instance/
.webassets-cache

# Scrapy stuff:
.scrapy

# Sphinx documentation
docs/_build/

# PyBuilder
.pybuilder/
target/

# Jupyter Notebook
.ipynb_checkpoints

# IPython
profile_default/
ipython_config.py

# pyenv
#   For a library or package, you might want to ignore these files since the
code is
#   intended to run in multiple environments; otherwise, check them in:
# .python-version

# pipenv
#   According to pypa/pipenv#598, it is recommended to include Pipfile.lock
in version control.
#   However, in case of collaboration, if having platform-specific
dependencies or dependencies
```

```
#   having no cross-platform support, pipenv may install dependencies that
don't work, or not
#   install all needed dependencies.
#Pipfile.lock

# UV
#   Similar to Pipfile.lock, it is generally recommended to include uv.lock
in version control.
#   This is especially recommended for binary packages to ensure
reproducibility, and is more
#   commonly ignored for libraries.
#uv.lock

# poetry
#   Similar to Pipfile.lock, it is generally recommended to include
poetry.lock in version control.
#   This is especially recommended for binary packages to ensure
reproducibility, and is more
#   commonly ignored for libraries.
#   https://python-poetry.org/docs/basic-usage/#commit-your-poetrylock-file-
to-version-control
#poetry.lock
#poetry.toml

# pdm
#   Similar to Pipfile.lock, it is generally recommended to include pdm.lock
in version control.
#   pdm recommends including project-wide configuration in pdm.toml, but
excluding .pdm-python.
#   https://pdm-project.org/en/latest/usage/project/#working-with-version-
control
#pdm.lock
#pdm.toml
.pdm-python
.pdm-build/

# pixi
#   Similar to Pipfile.lock, it is generally recommended to include
pixi.lock in version control.
#pixi.lock
#   Pixi creates a virtual environment in the .pixi directory, just like
venv module creates one
#   in the .venv directory. It is recommended not to include this directory
in version control.
```

```
.pixi

# PEP 582; used by e.g. github.com/David-OConnor/pyflow and github.com/pdm-
project/pdm
__pypackages__/

# Celery stuff
celerybeat-schedule
celerybeat.pid

# SageMath parsed files
*.sage.py

# Environments
.env
.envrc
.venv
env/
venv/
ENV/
env.bak/
venv.bak/

# Spyder project settings
.spyderproject
.spyproject

# Rope project settings
.ropeproject

# mkdocs documentation
/site

# mypy
.mypy_cache/
.dmypy.json
dmypy.json

# Pyre type checker
.pyre/

# pytype static type analyzer
.pytype/
```

```
# Cython debug symbols
cython_debug/

# PyCharm
#  JetBrains specific template is maintained in a separate
JetBrains.gitignore that can
#  be found at
https://github.com/github/gitignore/blob/main/Global/JetBrains.gitignore
#  and can be added to the global gitignore or merged into this file.  For a
more nuclear
#  option (not recommended) you can uncomment the following to ignore the
entire idea folder.
#.idea/

# Abstra
# Abstra is an AI-powered process automation framework.
# Ignore directories containing user credentials, local state, and settings.
# Learn more at https://abstra.io/docs
.abstra/

# Visual Studio Code
#  Visual Studio Code specific template is maintained in a separate
VisualStudioCode.gitignore
#  that can be found at
https://github.com/github/gitignore/blob/main/Global/VisualStudioCode.gitign
ore
#  and can be added to the global gitignore or merged into this file.
However, if you prefer,
#  you could uncomment the following to ignore the entire vscode folder
# .vscode/

# Ruff stuff:
.ruff_cache/

# PyPI configuration file
.pypirc

# Cursor
#  Cursor is an AI-powered code editor. `.cursorignore` specifies
files/directories to
#  exclude from AI features like autocomplete and code analysis. Recommended
for sensitive data
#  refer to https://docs.cursor.com/context/ignore-files
.cursorignore
```

```
.cursorindexingignore

# Marimo
marimo/_static/
marimo/_lsp/
__marimo__/
```